RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 – FUNDAMENTALS OF DATA SCIENCE LAB
Laboratory Record Note Book

Name : ……

Year / Branch / Section : ……

University Register No. : ……

College Roll No. : ……

Semester : ……

Academic Year : ……

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – FUNDAMENTALS OF DATA SCIENCE LAB

NAME	
ROLL NO	
YEAR	
SEC	

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……

Academic Year: …… Semester: …… Branch: ……

Register No. | |

Certified that this is the bonafide record of work done by the above student in the…… Laboratory during the academic year 2025 – 2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner

External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	https://github.com/EzhilvendanM20/FDS_LAB
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	

4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	https://github.com/EzhilvendanM20/FDS_LAB
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	
7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	

19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

Exp No:1.a Analyze the trend of data science job postings over the last Decade

Aim:
To Analyze the trend of data science job postings over the last Decade

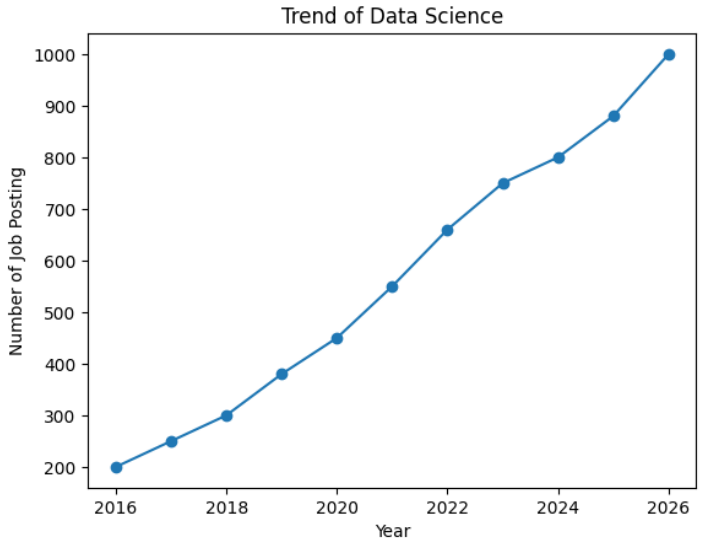
Description:
Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
'Year':list(range(2016,2027)),
'Job Posting':[200,250,300,380,450,550,660,750,800,880,1000]
}
df = pd.DataFrame(data)
plt.plot(df['Year'],df['Job Posting'],marker='o')
plt.xlabel("Year")
plt.ylabel("Number of Job Posting")
plt.title("Trend of Data Science")
plt.show()
```

Sample Data Input:
Year = 2016,2027
Job Posting = 200,250,300,380,450,550,660,750,800,880,1000

Sample Output:



Result:
Thus The Analyze the trend of data science job postings over the last Decade

Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Aim:
To Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

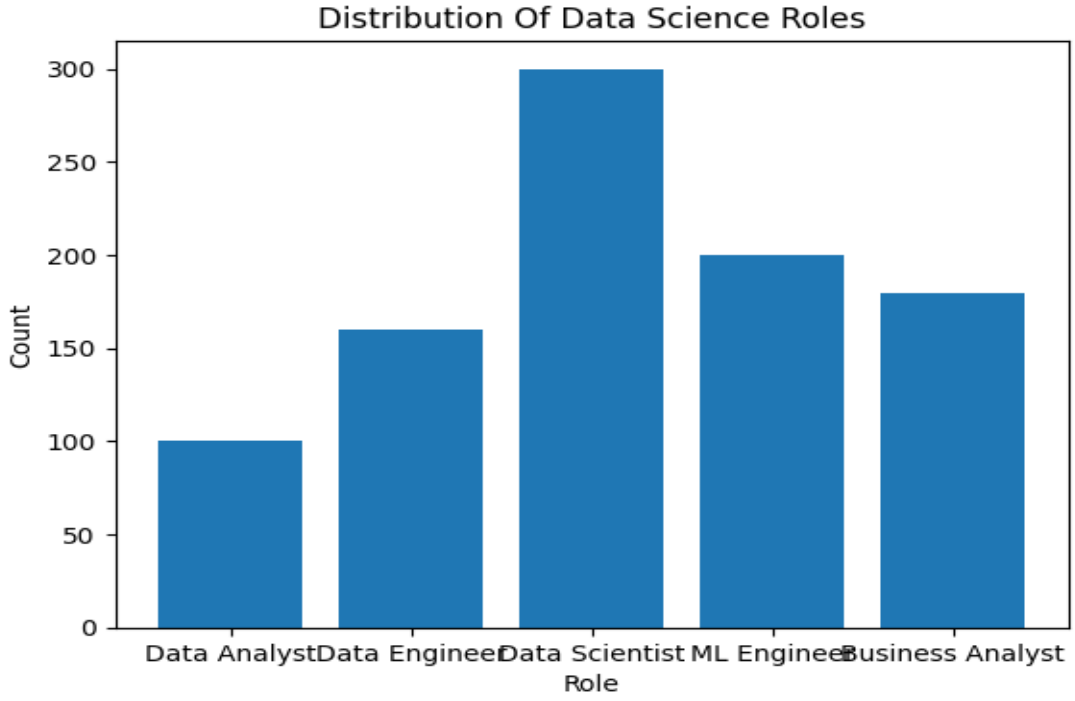
Description:
Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

Code:

```
import matplotlib.pyplot as plt
roles=['Data Analyst','Data Engineer','Data Scientist','ML Engineer','Business Analyst']
count = [100,160,300,200,180]
plt.bar(roles,count)
plt.xlabel("Role")
plt.ylabel("Count")
plt.title("Distribution Of Data Science Roles")
plt.show()
```

Sample Data Input:

roles=Data Analyst,Data Engineer,Data Scientist,ML Engineer,Business Analyst
count = 100,160,300,200,180



Result:
Thus The Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Aim:
To Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Description:
Create small datasets for each type and explain their characteristics.

Code:

```
import pandas as pd
data = {
   'ID':[1,2,3],
   'Name':['A','B','C'],
   'Age':[18,19,20]
}
structured_data=pd.DataFrame(data)
print('Structured Data:\n',structured_data)
unstructured_data="This is an example of Unstructured data.It can be a piece of text,an image,or a video file."
print('Unstructured Data:\n',unstructured_data)
semi_structured_data = {'ID':1,'Name':'D','Attributes':{'Height':185,'Weight':67}}
print('Semistructured:\n',semi_structured_data)
```

Output:

```
Structured Data:
   ID Name  Age
0   1    A   18
1   2    B   19
2   3    C   20
Unstructured Data:
 This is an example of Unstructured data.It can be a piece of text,an image,or a video file.
Semistructured:
 {'ID': 1, 'Name': 'D', 'Attributes': {'Height': 185, 'Weight': 67}}
```

Result:
Thus experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.

Aim:
To Conduct an experiment to encrypt and decrypt given sensitive data.

Description:
Use the cryptography library to encrypt and decrypt a piece of data.

Code:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
encrypt=f.encrypt(b"This Is 1st Experiment")
print('Encrypted : ',encrypt)
decrypt=f.decrypt(encrypt)
print('Decrypted : ',decrypt)
```

Output:
Encrypted :
b'gAAAAABqfbufPxHUArBHklsjI1aUfpMZIJYObCPxvWUenWbYToUlDHs7cY3465jfCEpPx-DLkVFMtRWyZ38nWo4re5kvqomxlbGtS0gGrz_0U6gqgL79mwQ='
Decrypted : b'This Is 1st Experiment'

Result:
Thus The **experiment to encrypt and decrypt given sensitive data**.

Exp No:2 Upload and Analyze the data set given in csv format and perform data.

Aim:
To Upload and Analyze the data set given in csv format and perform data.

Description: Use sample data set sales-data.csv.

Code:

```
from google.colab import files
uploaded = files.upload()
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import io
df = pd.read_excel(io.BytesIO(uploaded['SALES DATA.xlsx']))
print(df.head())
print(df.isnull().sum())
df['Sales'].fillna(df['Sales'].mean(),inplace=True)
df.dropna(subset=['Product','Quantity','Region'],inplace=True)
print(df.describe)

product_summary=df.groupby('Product').agg(
   {
      'Sales':'sum',
      'Quantity':'sum'
   }
).reset_index()
print(product_summary)
plt.figure(figsize=(10,6))
plt.bar(product_summary['Product'],product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales By Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales':'sum'}).reset_index()
plt.figure(figsize=(10,6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()

pivot_table=df.pivot_table(values='Sales',index='Region',columns='Product',aggfunc=np.sum,fill_value=
0)
print(pivot_table)

correlation_matrix=df.corr(numeric_only=True)
print(correlation_matrix)
import seaborn as sns
plt.figure(figsize=(8,6))
sns.heatmap(correlation_matrix,annot=True,cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

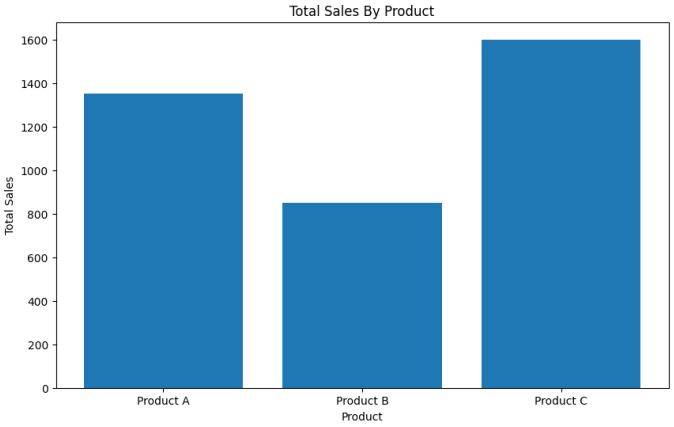
Output:

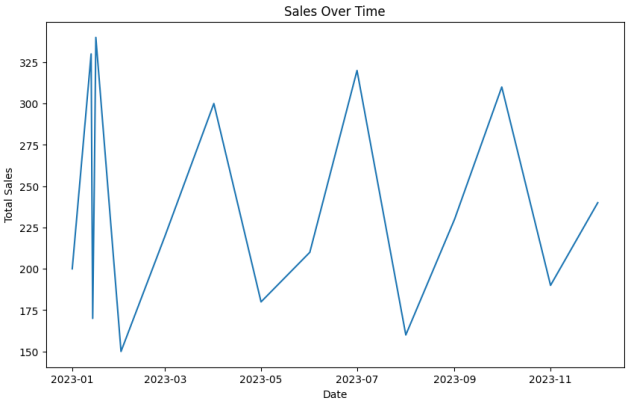
Date Product Sales Quantity Region

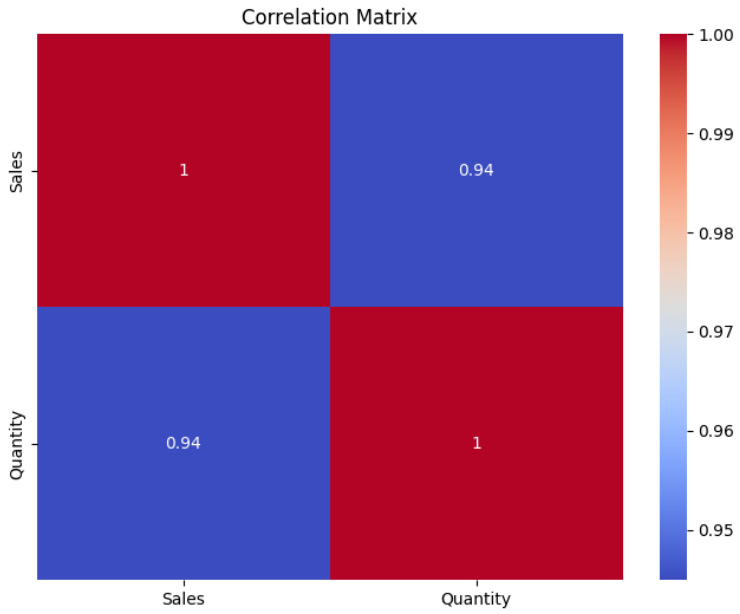
```
0  2023-01-01 00:00:00  Product A   200       4  North
1  2023-02-01 00:00:00  Product B   150       3  South
2  2023-03-01 00:00:00  Product A   220       5  North
3  2023-04-01 00:00:00  Product C   300       6   East
4  2023-05-01 00:00:00  Product B   180       4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
```

```
<bound method NDFrame.describe of                  Date    Product  Sales  Quantity Region
0   2023-01-01 00:00:00  Product A    200         4  North
1   2023-02-01 00:00:00  Product B    150         3  South
2   2023-03-01 00:00:00  Product A    220         5  North
3   2023-04-01 00:00:00  Product C    300         6   East
4   2023-05-01 00:00:00  Product B    180         4   West
5   2023-06-01 00:00:00  Product A    210         5  North
6   2023-07-01 00:00:00  Product C    320         7   East
7   2023-08-01 00:00:00  Product B    160         3  South
8   2023-09-01 00:00:00  Product A    230         6  North
9   2023-10-01 00:00:00  Product C    310         7   East
10  2023-11-01 00:00:00  Product B    190         4   West
11  2023-12-01 00:00:00  Product A    240         6  North
12           13-01-2023  Product C    330         8   East
13           14-01-2023  Product B    170         3  South
14           15-01-2023  Product A    250         7  North
15           16-01-2023  Product C    340         8   East>
```

```
  Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```







Result:

Thus The **Analyze the data set given in csv format and perform data.**

Exp No:3.a Conduct an experiment to show data visualization using line plot

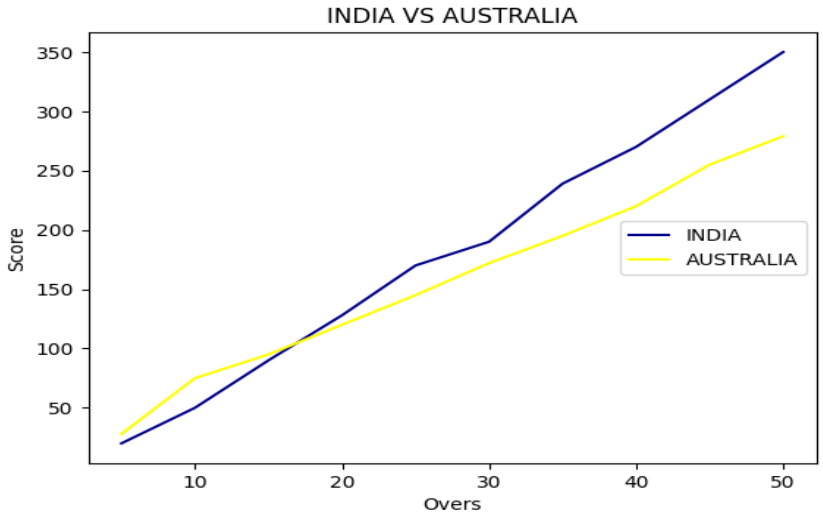
Aim:
To Conduct an experiment to show data visualization using line plot

Description: Take any sample data either through csv file or data fetched directly through code.

Code:

```
import matplotlib.pyplot as plt
Overs=list(range(5,51,5))
Indian_Score=[20,50,90,128,170,190,239,270,310,350]
Aus_Score=[28,75,95,120,145,172,195,220,255,279]
plt.plot(Overs,Indian_Score,color='darkblue',label='INDIA')
plt.plot(Overs,Aus_Score,color='yellow',label='AUSTRALIA')
plt.title('INDIA VS AUSTRALIA')
plt.xlabel('Overs')
plt.ylabel('Score')
plt.legend()
plt.legend(loc='center right')
plt.show()
```

Output:



Result:
Thus an experiment to show data visualization using line plot

Exp No:3.b Conduct an experiment to show data visualization using bar chart.

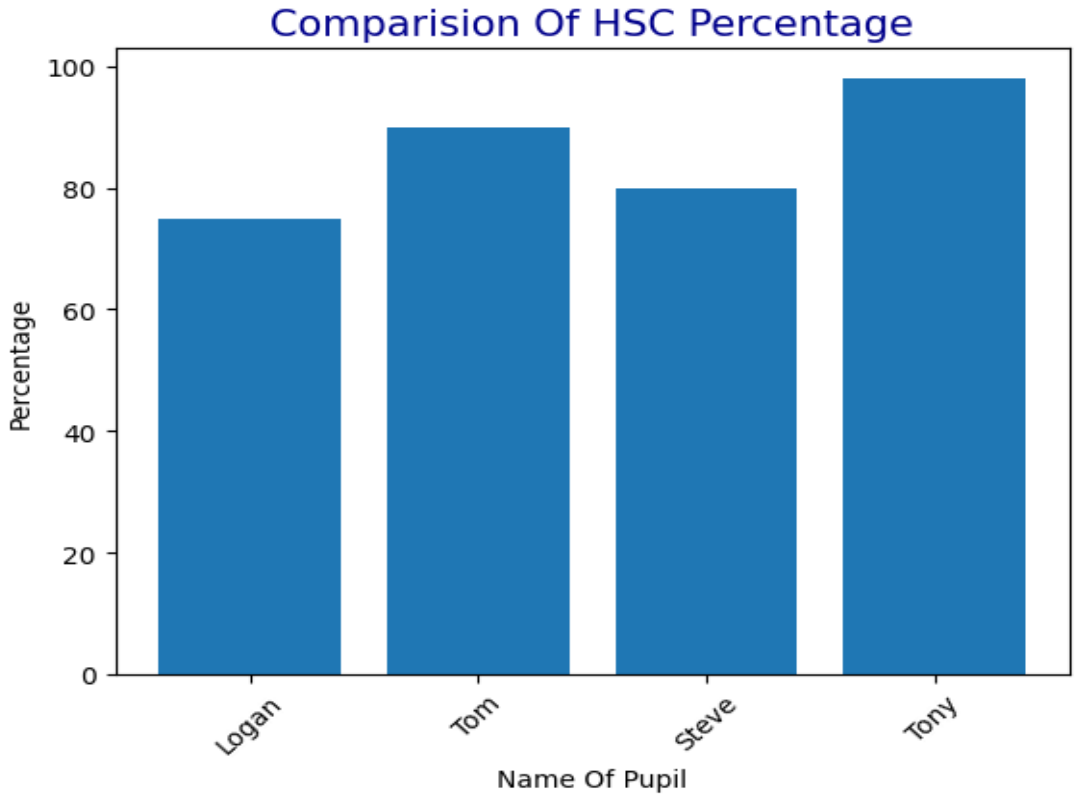
Aim:
To Conduct an experiment to show data visualization using bar chart.

Description: Take any sample data either through csv file or data fetched directly through code.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
Names = ['Logan','Tom','Steve','Tony']
axis = np.arange(len(Names))
percent = [75,90,80,98]
plt.bar(Names,percent)
plt.xticks(axis,Names,rotation = 45)
plt.xlabel('Name Of Pupil')
plt.ylabel('Percentage')
plt.title('Comparision Of HSC Percentage',fontsize = 15,color='darkBlue')
plt.show()
```

Output:



Result:

Thus an experiment to show data visualization using bar chart.

Exp No:3.c Conduct an experiment to show data visualization using pie chart.

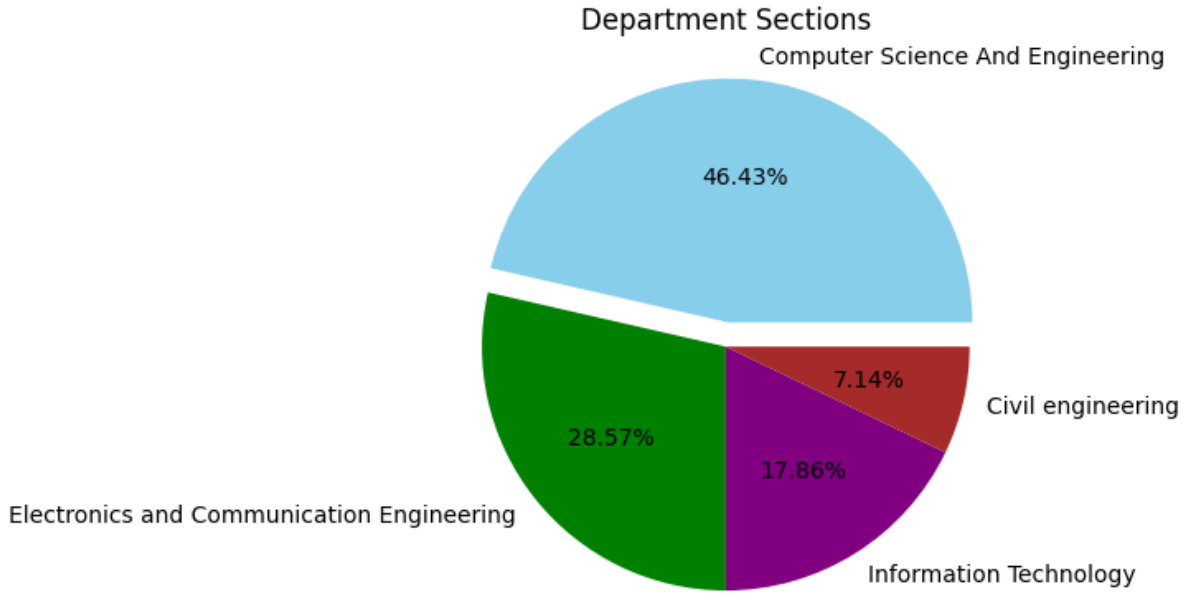
Aim:
To Conduct an experiment to show data visualization using pie chart.

Description: Take any sample data either through csv file or data fetched directly through code.

Code:

```
import matplotlib.pyplot as plt
label=['Computer Science And Engineering','Electronics and Communication Engineering','Information Technology','Civil engineering']
sections=[13,8,5,2]
color=['skyblue','green','purple','brown']
explode1=[0.1,0,0,0]
plt.title('Department Sections')
plt.pie(sections,labels=label,colors=color,explode=explode1,autopct='%0.2f%%')
plt.show()
```

Output:



Result:
Thus an experiment to show data visualization using pie chart.

Exp No:4 To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Aim:
To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Description: Import the word_tokenize function and gutenberg.

Code:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample = gutenberg.raw('austen-emma.txt')
token = word_tokenize(sample)
wlist = []
for i in range(50):
  wlist.append(token[i])
word_freq=[]
for i in wlist:
  word_freq.append(wlist.count(i))
print('Pairs:\n'+str(list(zip(wlist,word_freq))))
```

Output:

[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever', 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposition', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('existence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world', 1), ('with', 2)]

Result:

Thus The Count the frequency of occurrence of a word in a body of text is often needed during text processing.

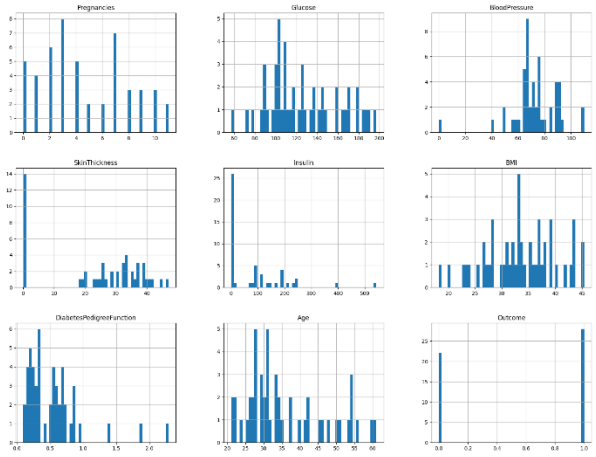
Exp No:5 Data Collection and Initial Exploration

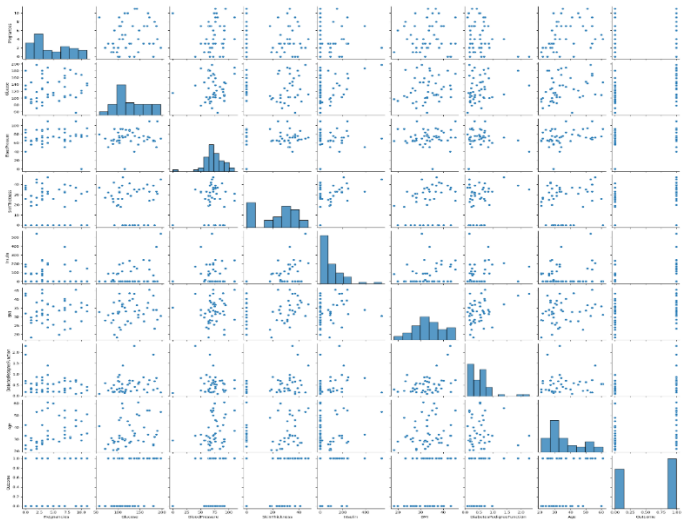
Aim:
To collect, load, and perform initial exploration of the diabetes dataset.

Materials Needed:
□ Diabetes dataset (available from sources such as UCI Machine Learning Repository or Kaggle)
□ Python with libraries: pandas, numpy, matplotlib, seaborn
Procedure and Code:
1. Data Collection:
db = pd.read_csv("diabetes_dataset_50.csv")
db.head()

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

2. Initial Exploration:
print(db.info)
print(db.describe)
import matplotlib.pyplot as plt
import seaborn
db.hist(bins=50,figsize=(20,15))
plt.show()
import seaborn as sns
sns.pairplot(db)
plt.show()





Result:

Thus the diabetes dataset collected, loaded and perform initial exploration of the diabetes dataset

Exp No: 6 Time Series Analysis of Website Traffic
Aim:
To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

Dataset:
A dataset containing daily website traffic data with columns such as Date, Page_Views, Unique_Visitors, and Bounce_Rate**.**

Methodology:
1. Load and Clean Data:
o Import the dataset and parse the Date column as a datetime object.
o Handle missing values by interpolation.
2. Time Series Decomposition:
o Decompose the time series data of Page_Views to identify trend, seasonality, and residuals.
3. Visualization:
o Plot the time series data for Page_Views, Unique_Visitors, and Bounce_Rate.
o Use moving averages to smooth the time series and highlight trends.
o Create seasonal plots to visualize patterns over time.
4. Anomaly Detection:
o Identify and visualize anomalies in the time series data using statistical methods such as the Z-score.

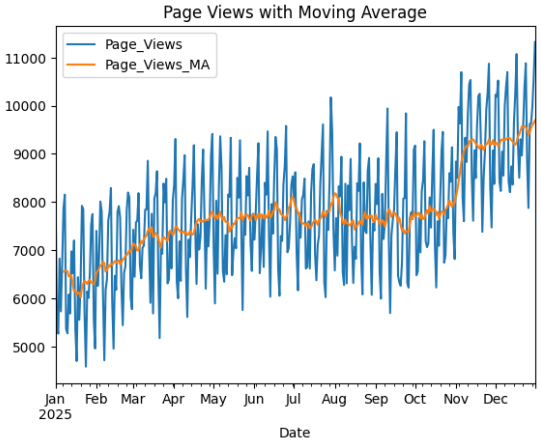
Sample code:

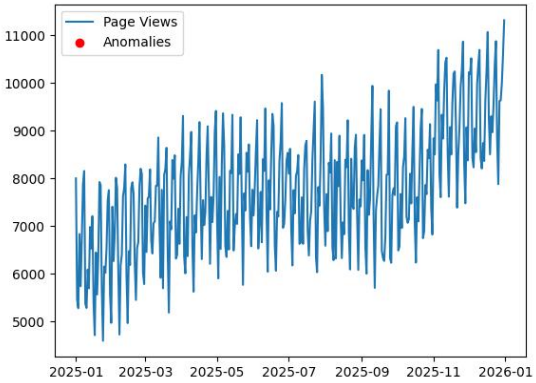
```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
data = pd.read_csv("website_traffic (1).csv",parse_dates=['Date'],index_col='Date')
decomposition = seasonal_decompose(
   data['Page_Views'],
   model='additive',
   period=7
)
decomposition.plot()
plt.show()
data['Page_Views_MA'] = data['Page_Views'].rolling(window=7).mean()

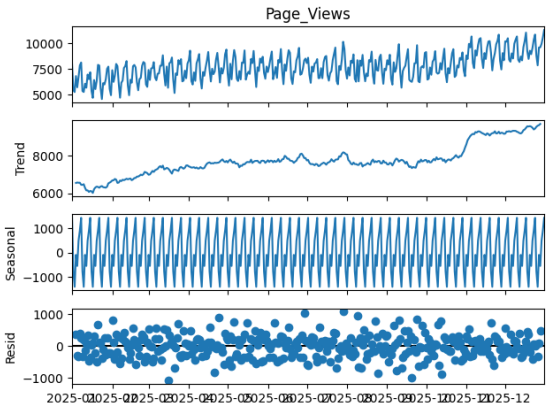
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()
data['Page_Views_Z'] = (
   (data['Page_Views'] - data['Page_Views'].mean())
   / data['Page_Views'].std()
)
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(
   data.index,
   data['Page_Views'],
   label='Page Views'
)

plt.scatter(
   anomalies.index,
   anomalies['Page_Views'],
   color='red',
   label='Anomalies'
)
plt.legend()
```

plt.show(


Page Views with Moving Average
Page_Views
Page_Views_MA
11000
10000
9000
8000
7000
6000
5000
Jan
2025
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Date


Page Views
Anomalies
11000
10000
9000
8000
7000
6000
5000
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11
2026-01


Page_Views
10000
7500
5000
Trend
8000
6000
Seasonal
1000
0
−1000
Resid
1000
0
−1000

Result :
Thus Analysis of the time series data of website traffic has identified trends , seasonality and anomalies.

Exp No: 7 Random Sampling and Sampling Distribution

Aim:

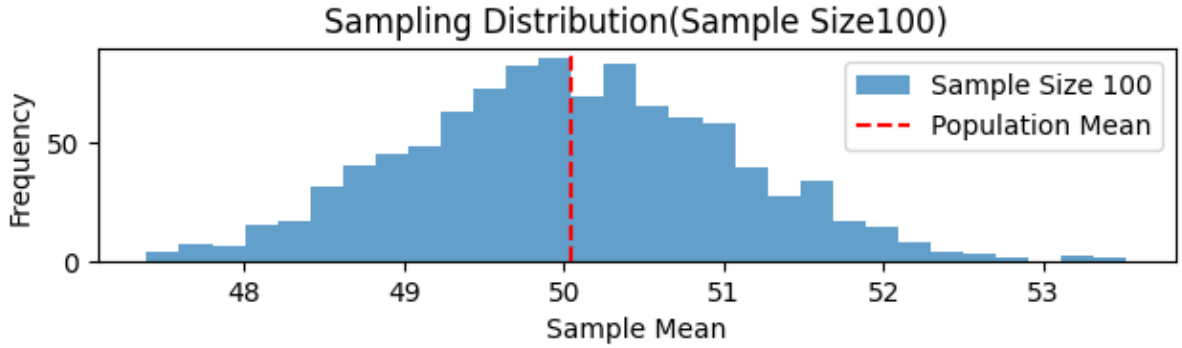
To explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

Steps:

1. Generate a Population:
   ○ Create a population of data with a specified distribution (e.g., normal distribution).
2. Random Sampling:
   ○ Perform random sampling from the population to create multiple samples of different sizes.
   ○ Compute sample statistics (mean, standard deviation, etc.) for each sample.
3. Sampling Distribution:
   ○ Plot histograms or density plots of sample statistics (e.g., sample means).
   ○ Compare the sampling distribution of the sample statistic (mean) with the population distribution.
4. Central Limit Theorem (Optional):
   ○ Demonstrate the Central Limit Theorem by showing that as sample size increases, the sampling distribution of the sample mean approaches a normal distribution regardless of the population distribution.

Example Code:

```
import numpy as np
import matplotlib.pyplot as plt
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean,population_std,population_size)
sample_size = [30,50,100]
num_samples = 1000
sample_means = {}
for size in sample_size:
  sample_means[size]=[]
  for _ in range(num_samples):
    sample = np.random.choice(population,size=size,replace=False)
    sample_means[size].append(np.mean(sample))
plt.figure(figsize=(12,8))
for i,size in enumerate(sample_size):
  plt.subplot(len(sample_size),1,i+1)
  plt.hist(sample_means[size],bins=30,alpha=0.7,label=f'Sample Size {size}')
  plt.axvline(np.mean(population),color='red',linestyle='dashed',linewidth=1.5,label="Population Mean")
  plt.title(f'Sampling Distribution(Sample Size{size})')
  plt.xlabel('Sample Mean')
  plt.ylabel('Frequency')
  plt.legend()
  plt.tight_layout()
  plt.show()
```



Result:

Thus the concept of random sampling and sampling distribution understand by using python in jupyter notebook.

Exp No: 8 Conduct Z test for the Data Given using Python Objective:

Aim:

To test whether the average weight of a species of birds differs from 150 grams.

Procedure:

1. Null Hypothesis (H0): The average weight of the birds is 150 grams.
2. Alternative Hypothesis (H1): The average weight of the birds is not 150 grams.
3. Sample: Measure the weights of 30 birds randomly selected from the population.
4. Z-Test: Conduct a Z-test to compare the sample mean to 150 grams.
5. Decision Rule: Use a significance level of $\alpha = 0.05$.

Python Code Example:

```
import numpy as np
import scipy.stats as stats
sample_data =
np.array([152,148,151,149,147,153,150,148,152,149,151,150,149,152,151,148,150,152,149,150,148,153,151,1
50,149,152,148,151,150,153])
population_mean =150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data,ddof=1)
n=len(sample_data)
z_statistic = (sample_mean - population_mean)/(sample_std/np.sqrt(n))
p_value = 2*(1-stats.norm.cdf(np.abs(z_statistic)))
print(f'Sample Mean: {sample_mean:.2f}')
print(f'z_statistic: {z_statistic:.4f}')
print(f'P-Value: {p_value:.4f}')
alpha=0.05
if p_value<alpha:
  print('Reject the null hypothesis: The average weight is significantly different from 150 grams.')
else:
  print('Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.')
```

Output:

Sample Mean: 150.20
z_statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.

Result:

Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.

Exp No: 9 Experiment to understand Matplotlib library use cases in Data Science through visualization

Description: Use Iris data set to understand the Matplot lib library

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
df = iris.frame
#print(df.shape)
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
df.describe()
```

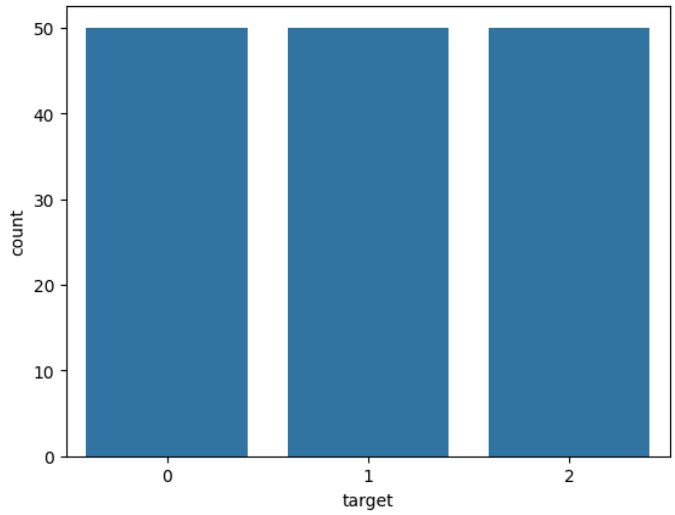
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```
df.columns
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'target'],
      dtype='object')
df.value_counts('target')
```

	count
target	
0	50
1	50
2	50

dtype: int64

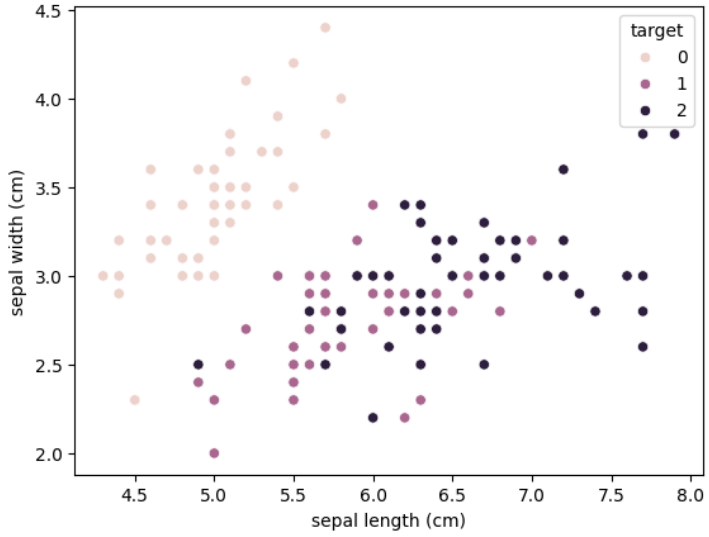
```
sns.countplot(x='target',data=df)
plt.show()
```



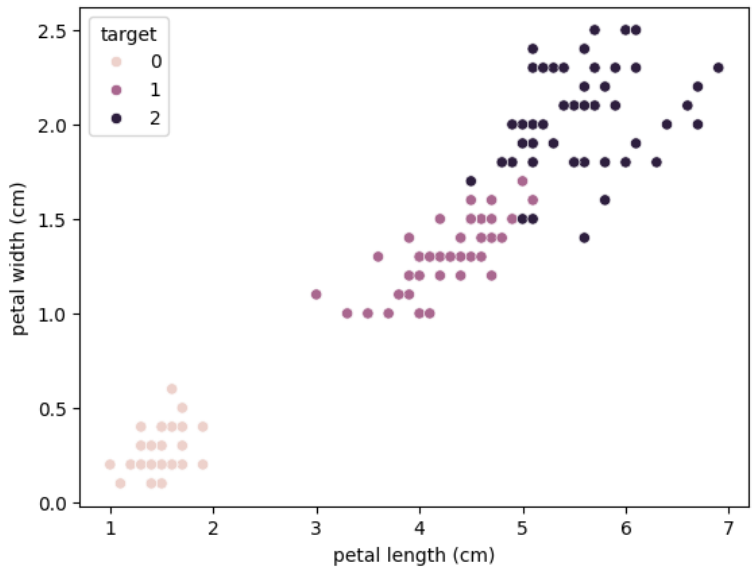

```
dummies = pd.get_dummies(df.target)
finaldataset = pd.concat([pd.get_dummies(df.target),df.iloc[:,[0,1,2,3]]],axis=1)
finaldataset.head()
```

	0	1	2	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	True	False	False	5.1	3.5	1.4	0.2
1	True	False	False	4.9	3.0	1.4	0.2
2	True	False	False	4.7	3.2	1.3	0.2
3	True	False	False	4.6	3.1	1.5	0.2
4	True	False	False	5.0	3.6	1.4	0.2

```
sns.scatterplot(x='sepal length (cm)',y='sepal width (cm)',hue = 'target',data=df)
<Axes: xlabel='sepal length (cm)', ylabel='sepal width (cm)'>
```

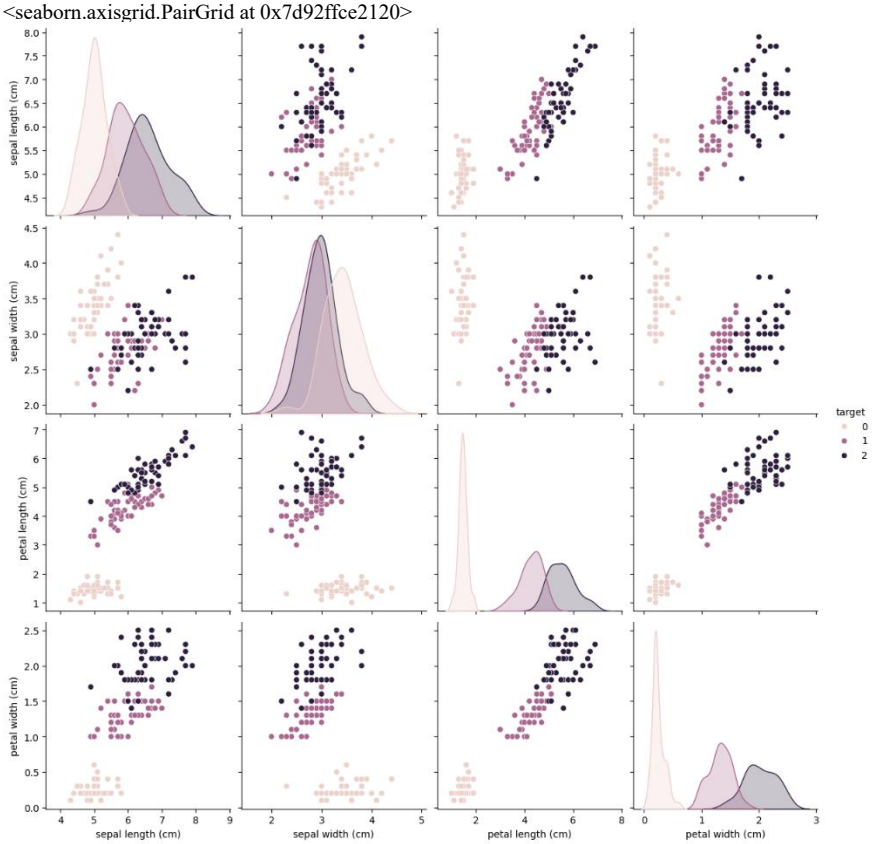



```
sns.scatterplot(x='petal length (cm)',y='petal width (cm)',hue = 'target',data=df)
<Axes: xlabel='petal length (cm)', ylabel='petal width (cm)'>
```

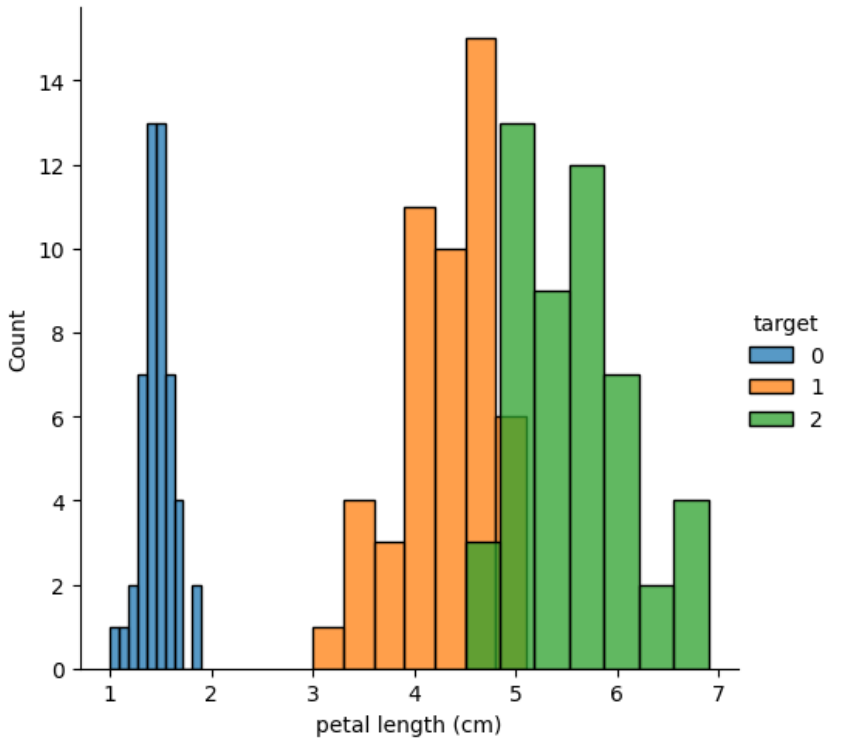



```
sns.pairplot(df,hue='target',height=3)
```

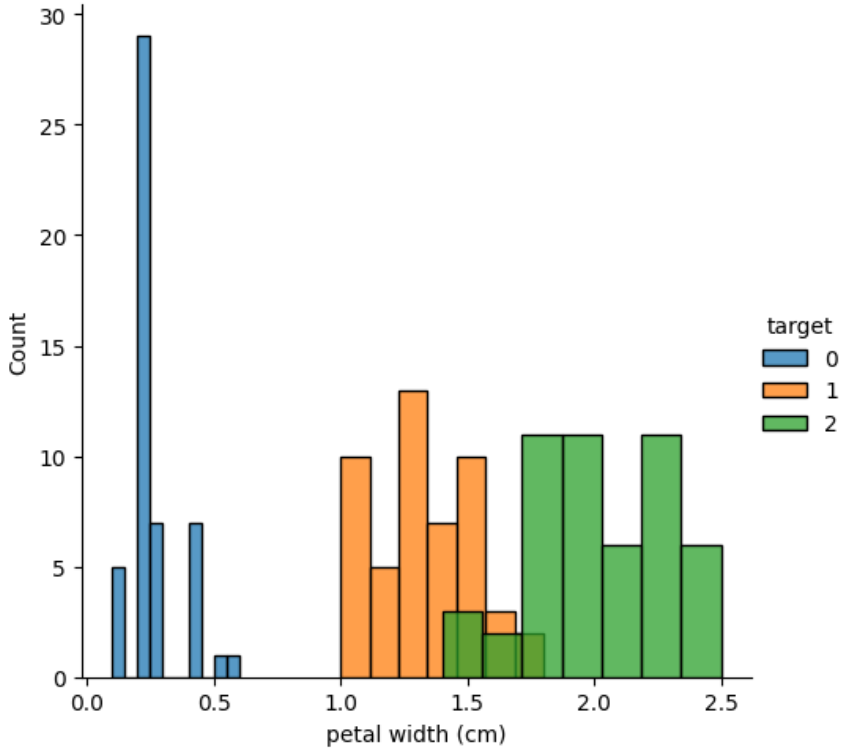
```
<seaborn.axisgrid.PairGrid at 0x7d92ffce2120>
```



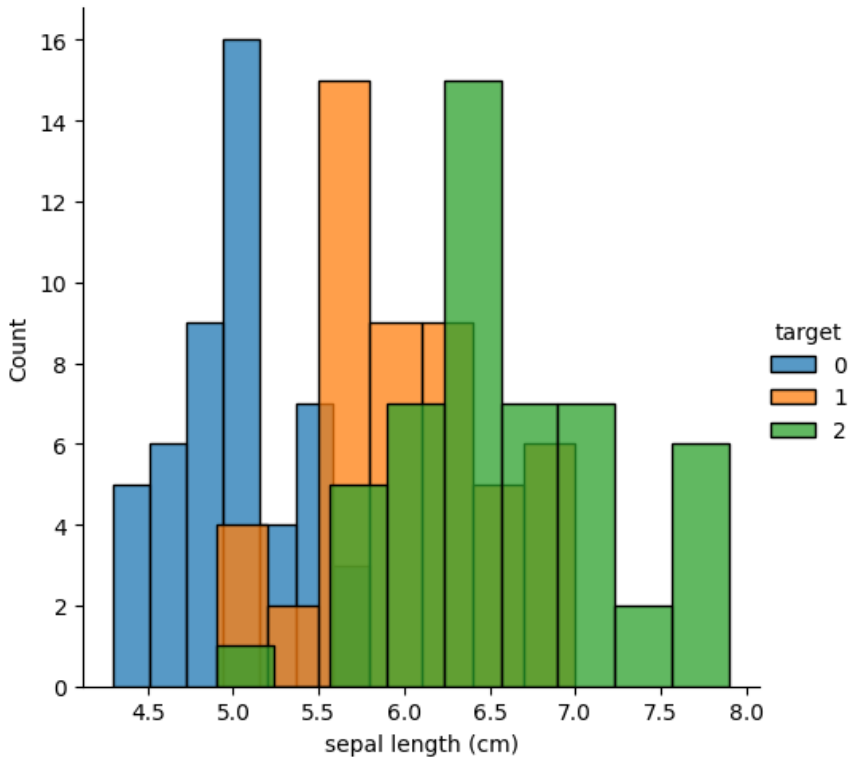

```
sns.FacetGrid(df,hue='target',height=5).map(sns.histplot,'petal length (cm)').add_legend()
plt.show()
```



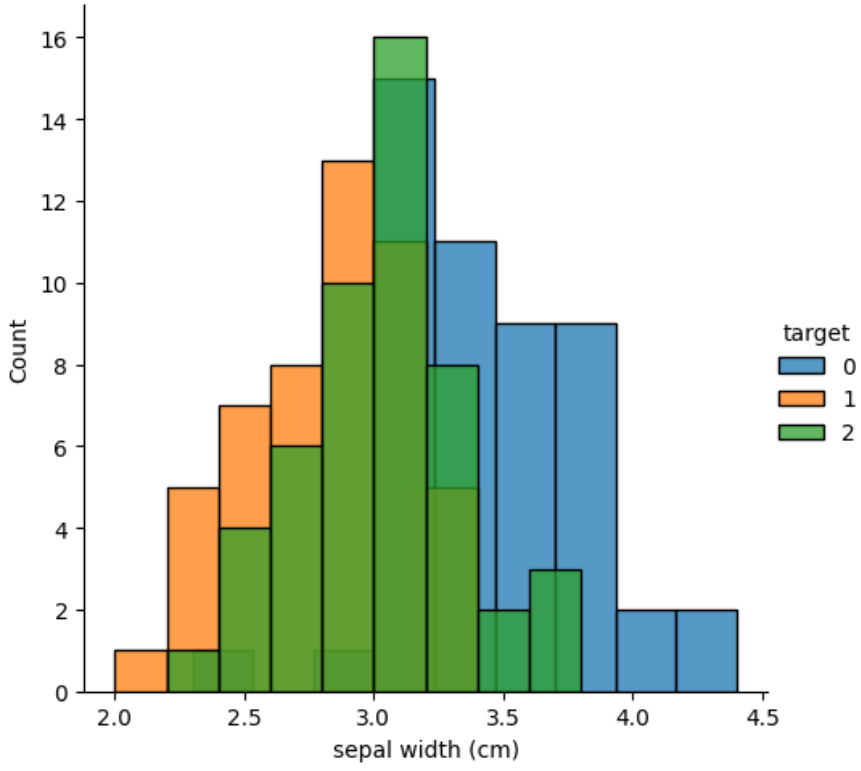

```
sns.FacetGrid(df,hue='target',height=5).map(sns.histplot,'petal width (cm)').add_legend()
plt.show()
```




```
sns.FacetGrid(df,hue='target',height=5).map(sns.histplot,'sepal length (cm)').add_legend()
plt.show()
```




```
sns.FacetGrid(df,hue='target',height=5).map(sns.histplot,'sepal width (cm)').add_legend()
plt.show()
```



Result:
Iris data set has understood by using the Matplot library

Exp No:10 Experiment to understand array function in Data science.

Description:

Understand array function using Numpy library

Code:

```
import numpy as np
array = np.random.randint(1,100,9)
array
array([56, 85, 37, 25,  8, 71, 11, 14, 47])
np.sqrt(array)
array([9.48683298, 5.09901951, 6.63324958, 6.63324958, 5.56776436,
       5.47722558, 9.48683298, 9.94987437, 6.4807407 ])

new_array = array.reshape(3,3)
1

new_array
array([[56, 85, 37],
       [25,  8, 71],
       [11, 14, 47]])

new_array.ndim
2

new_array.ravel()
array([56, 85, 37, 25,  8, 71, 11, 14, 47])

newm = new_array.reshape(3,3)
newm

array([[56, 85, 37],
       [25,  8, 71],
       [11, 14, 47]])

newm[2,1:3]
array([14, 47])

newm[1:2,1:3]
array([[ 8, 71]])

new_array[0:3,0:0]
array([], shape=(3, 0), dtype=int64)

new_array[0:2,0:1]
array([[56],
       [25]])

new_array[0:3,0:1]
array([[56],
       [25],
```

```
       [11]])

new_array[1:3]
array([[25,  8, 71],
       [11, 14, 47]])
```

Result:

Thus Understood array function using Numpy library

Exp No:11 Experiment to understand pandas library use cases in Data science.

Description: Understand data frame use cases using pandas library

Code:

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
df
```

	0	1	2
0	1	Smith	50000
1	2	Jones	60000

```
df.columns=['Empd','Name','Salary']
df
```

	Empd	Name	Salary
0	1	Smith	50000
1	2	Jones	60000

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
df = pd.read_csv('50_Startups.csv')
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
df.head()
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

```
df.tail()
```

	R&D Spend	Administration	Marketing Spend	State	Profit
45	1000.23	124153.04	1903.93	New York	64926.08
46	1315.11	110716.75	197661.10	Florida	42559.73
47	0.00	154146.00	0.00	California	35673.41
48	542.05	51743.15	0.00	New York	35673.41
49	0.00	116983.80	45173.06	California	14681.40

df= pd.read_csv('/content/employee.csv')
df.head()

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000

df.tail()

	emp id	name	salary
5	6	SRIHARSHA V	6500
6	7	SRINITHI S	5500
7	8	SRIVATSAN R	7500
8	9	SUBHASHINI M	8500
9	10	SUDHARSHAN R	9000

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  10 non-null     int64
 1   name    10 non-null     object
 2   salary  10 non-null     int64
dtypes: int64(2), object(1)
memory usage: 372.0+ bytes
```

df.salary

	salary
0	5000
1	6000
2	7000
3	5000
4	8000
5	6500
6	5500
7	7500
8	8500
9	9000

dtype: int64
type(df.salary)

df.salary.mean()
np.float64(6800.0)
df.salary.median()
6750.0
df.salary.var()
2066666.6666666667
df.salary.std()
1437.5905768565217
df.describe()

	emp id	salary
count	10.00000	10.000000
mean	5.50000	6800.000000
std	3.02765	1437.590577
min	1.00000	5000.000000
25%	3.25000	5625.000000
50%	5.50000	6750.000000
75%	7.75000	7875.000000
max	10.00000	9000.000000

df.describe(include='all')

	emp id	name	salary
count	10.00000	10	10.000000
unique	NaN	10	NaN
top	NaN	SREE VARSSINI K S	NaN
freq	NaN	1	NaN
mean	5.50000	NaN	6800.000000
std	3.02765	NaN	1437.590577
min	1.00000	NaN	5000.000000
25%	3.25000	NaN	5625.000000
50%	5.50000	NaN	6750.000000
75%	7.75000	NaN	7875.000000
max	10.00000	NaN	9000.000000

empCol=df.columns
empCol
Index(['emp id', 'name', 'salary'], dtype='object')
emparray=df.values
emparray
array([[1, 'SREE VARSSINI K S', 5000],
[2, 'SREEMATHI B', 6000],
[3, 'SREYA G', 7000],
[4, 'SREYASKARI MULLAPUDI', 5000],
[5, 'SRI AKASH U G', 8000],
[6, 'SRIHARSHA V', 6500],
[7, 'SRINITHI S', 5500],
[8, 'SRIVATSAN R', 7500],
[9, 'SUBHASHINI M', 8500],
[10, 'SUDHARSHAN R', 9000]], dtype=object)
employee_DF=pd.DataFrame(emparray,columns=empCol)
employee_DF

	emp id	name	salary
0	1	SREE VARSSINI K S	5000

	emp id	name	salary
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRIHARSHA V	6500
6	7	SRINITHI S	5500
7	8	SRIVATSAN R	7500
8	9	SUBHASHINI M	8500
9	10	SUDHARSHAN R	9000

Result:

Understood data frame use cases using pandas library

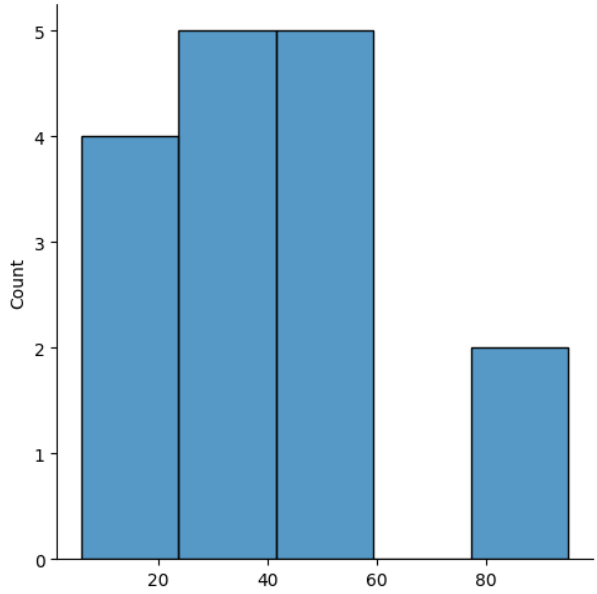
Exp No:12 Experiment to detect outliers in a given data set.

Description: Understand the procedure to identify the outliers in a given dataset

Code:

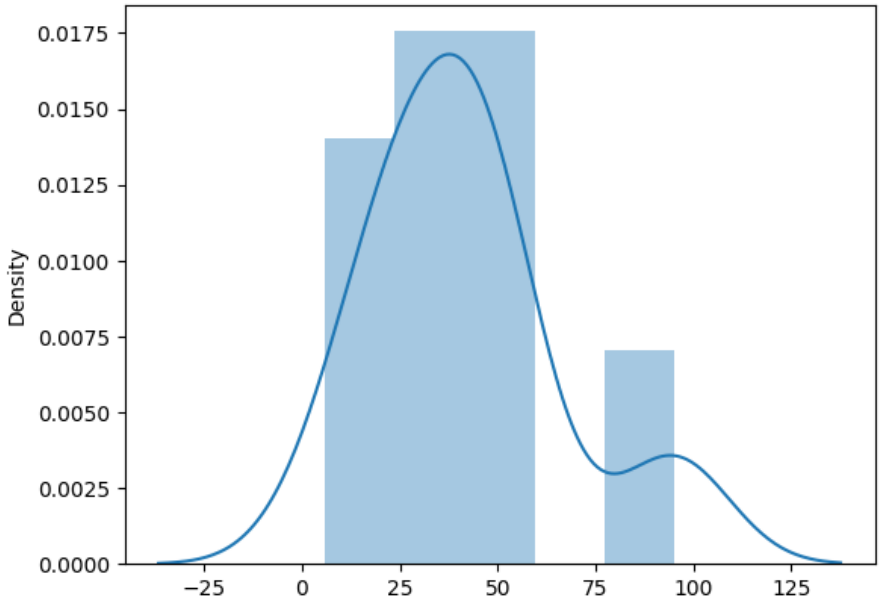
```
import numpy as np
array=np.random.randint(1,100,16)
array
array([41, 25, 37, 46, 55, 47, 16, 95, 47,  6, 23, 49, 26, 23, 40, 95])
array.mean()
np.float64(41.9375)
np.percentile(array,25)
np.float64(24.5)
np.percentile(array,50)
np.float64(40.5)
np.percentile(array,75)
np.float64(47.5)
np.percentile(array,100)
np.float64(95.0)
def outDetection(array):
 sorted(array)
 Q1,Q3=np.percentile(array,[25,75])
 IQR=Q3-Q1
 lr=Q1-(1.5*IQR)
 ur=Q3+(1.5*IQR)
 return lr,ur
lr,ur=outDetection(array)
lr,ur
(np.float64(-10.0), np.float64(82.0))
import seaborn as sns
%matplotlib inline
sns.displot(array)
```

```
<seaborn.axisgrid.FacetGrid at 0x7e4064b00ad0>
```

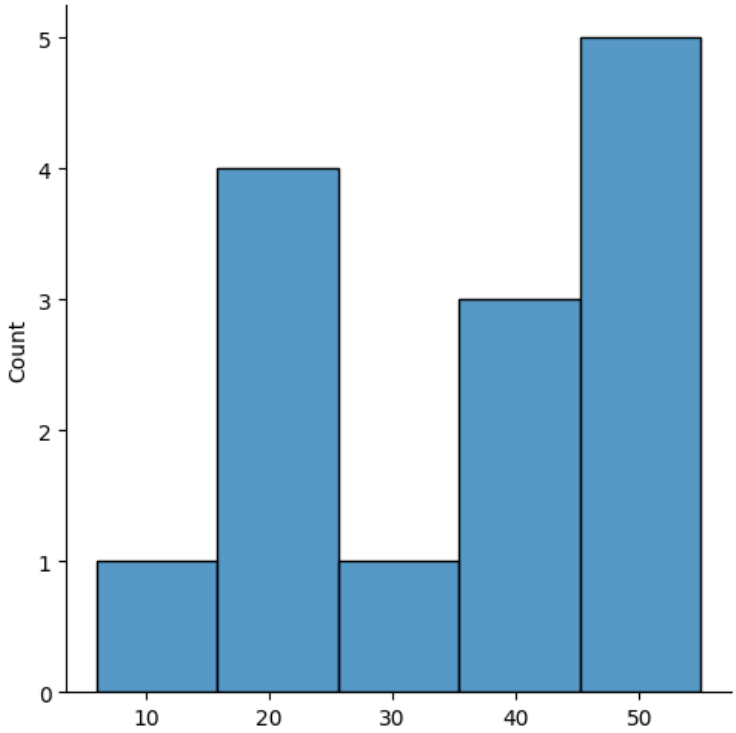



```
sns.distplot(array)
```

```
/tmp/ipykernel_820/1133588802.py:1: UserWarning:
`distplot` is a deprecated function and will be removed in seaborn v0.14.0
Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).
For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
  sns.distplot(array)
<Axes: ylabel='Density'>
```



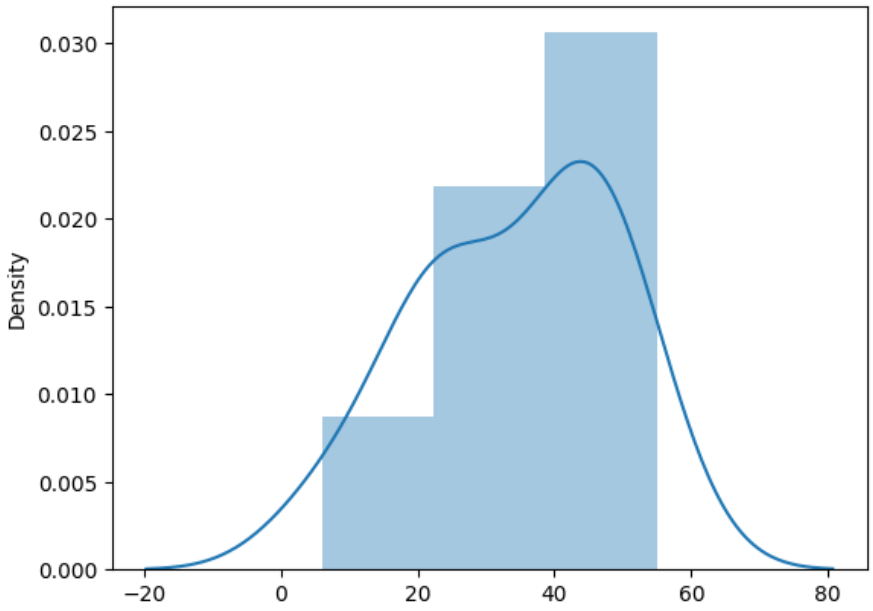
new_array=array[(array>lr) & (array<ur)]
new_array
array([41, 25, 37, 46, 55, 47, 16, 47, 6, 23, 49, 26, 23, 40])
sns.displot(new_array)
<seaborn.axisgrid.FacetGrid at 0x7e406494f890>



lr1,ur1=outDetection(new_array)
lr1,ur1
(np.float64(-11.375), np.float64(81.625))
final_array=new_array[(new_array>lr1) & (new_array<ur1)]
final_array
array([41, 25, 37, 46, 55, 47, 16, 47, 6, 23, 49, 26, 23, 40])
sns.distplot(final_array)
/tmp/ipykernel_820/209491988.py:1: UserWarning:
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).
For a guide to updating your code to use the new functions, please see

https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

```
  sns.distplot(final_array)
<Axes: ylabel='Density'>
```



Result:

Thus, Unserstood the procedure to identify the outliers in a given dataset

Exp No:13 Experiment to handle missing values and inappropriate data in a given data set.

Description: Understand the procedure to handle missing and inappropriate data

Aim: Demonstrate an experiment to handle missing data and inappropriate data in a Data set using Python Pandas Library for Data Preprocessing.

Dataset Given:
Hotel.csv
About Dataset:
No.of Columns =9 (called as series – CustomerID, Age_Group, Rating(1-5),Hotel, FoodPreference, Bill, NoOfPax, EstimatedSalary)
CutomerID: Numerical Continuous data
Age: Categorical Data
Rating (1-5): Numerical Discrete Data
Hotel: Categorical Data
Food: Categorical Data
Bill: Numerical Continuous data
NoOfPax: Numerical Discrete
EstimatedSalary: Numerical Continuous data

Python Code:

```
import numpy as np
import pandas as pd
df = pd.read_csv('/content/hotel_data.csv')
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

```
df.duplicated()
```

	0
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	True
10	False

```
df.drop_duplicates()
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

```
df.isnull()
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
5	False	False	False	False	False	False	False	False	False
6	False	False	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False	False	False
10	False	False	False	False	False	False	False	False	False

```
df.isnull().sum()
```

	0
CustomerID	0
Age_Group	0
Rating(1-5)	0
Hotel	0
FoodPreference	0
Bill	0
NoOfPax	0
EstimatedSalary	0
Age_Group.1	0

	0
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	True
10	False

dtype: bool
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 # Column Non-Null Count Dtype
--- ------ -------------- -----
 0 CustomerID 11 non-null int64
 1 Age_Group 11 non-null object
 2 Rating(1-5) 11 non-null int64
 3 Hotel 11 non-null object
 4 FoodPreference 11 non-null object
 5 Bill 11 non-null int64
 6 NoOfPax 11 non-null int64
 7 EstimatedSalary 11 non-null int64
 8 Age_Group.1 11 non-null object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
df.drop_duplicates(inplace=True)
df

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

len(df)

10

index=np.array(list(range(0,len(df))))

df.set_index(index,inplace=True)

index

array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

df

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

df.drop(['Age_Group.1'],axis=1,inplace=True)

df

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1	20-25	4	Ibis	veg	1300	2	40000
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000
2	3	25-30	6	RedFox	Veg	1322	2	30000
3	4	20-25	-1	LemonTree	Veg	1234	2	120000
4	5	35+	3	Ibis	Vegetarian	989	2	45000
5	6	35+	3	Ibys	Non-Veg	1909	2	122220
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122
7	8	20-25	7	LemonTree	Veg	2999	-10	345673
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777

df.CustomerID.loc[df.CustomerID<0]=np.nan

df.Bill.loc[df.Bill<0]=np.nan

df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan

```
df
df.Age_Group.unique()
array(['20-25', '30-35', '25-30', '35+'], dtype=object)
df.Hotel.unique()
array(['Ibis', 'LemonTree', 'RedFox', 'Ibys'], dtype=object)
df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
df.FoodPreference.unique
df.FoodPreference.replace(['Vegetarian','veg'],'Veg',inplace=True)
df.FoodPreference.replace(['non-Veg'],'Non-Veg',inplace=True)
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df.Bill.fillna(round(df.Bill.mean()),inplace=True)
df
```

Result:

Thus, Understood The experiment to handle missing data and inappropriate data in a Data set using Python Pandas Library for Data Preprocessing.

Exp No:14 Experiment to understand feature scaling.

Description: Understand the importance of feature scaling

Code:

```
import pandas as pd
df = pd.read_csv('/content/pre-process_datasample.csv')
df.head()
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
features=df.iloc[:,:-1].values
```

/tmp/ipykernel_1268/3424832005.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
  df.Country.fillna(df.Country.mode()[0],inplace=True)

```
label=df.iloc[:,-1].values
from sklearn.impute import SimpleImputer
import numpy as np
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
Salary.fit(features[:,[2]])
SimpleImputer()
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
features
```

```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['Germany', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

```
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
final_set
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [0.0, 1.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
feat_standard_scaler
array([[ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
feat_minmax_scaler
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [0.        , 1.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

Result:

Understood the importance of feature scaling

Exp No:15 Experiment to understand the data preprocessing in Data science
Description: Understand the importance of Data preprocessing in data science

Code:

```
import numpy as np
import pandas as pd
df = pd.read_csv('/content/pre-process_datasample.csv')
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
df.Country.mode()
```

	Country
0	France

dtype: object

```
df.Country.mode()[0]
```

```
'France'
```

```
type(df.Country.mode())
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
df
pd.get_dummies(df.Country)
```

	France	Germany	Spain
0	True	False	False
1	False	False	True
2	False	True	False
3	False	False	True
4	False	True	False
5	True	False	False

	France	Germany	Spain
6	False	False	True
7	True	False	False
8	False	True	False
9	True	False	False

updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],axis=1)
updated_dataset

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	No
1	False	False	True	27.0	48000.0	Yes
2	False	True	False	30.0	54000.0	No
3	False	False	True	38.0	61000.0	No
4	False	True	False	40.0	63778.0	Yes
5	True	False	False	35.0	58000.0	Yes
6	False	False	True	38.0	52000.0	No
7	True	False	False	48.0	79000.0	Yes
8	False	True	False	50.0	83000.0	No
9	True	False	False	37.0	67000.0	Yes

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        10 non-null     float64
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

Result:

Understood the importance of Data preprocessing in data science

Exp No:16 Experiment to understand EDA-Quantitative and Qualitative analysis.

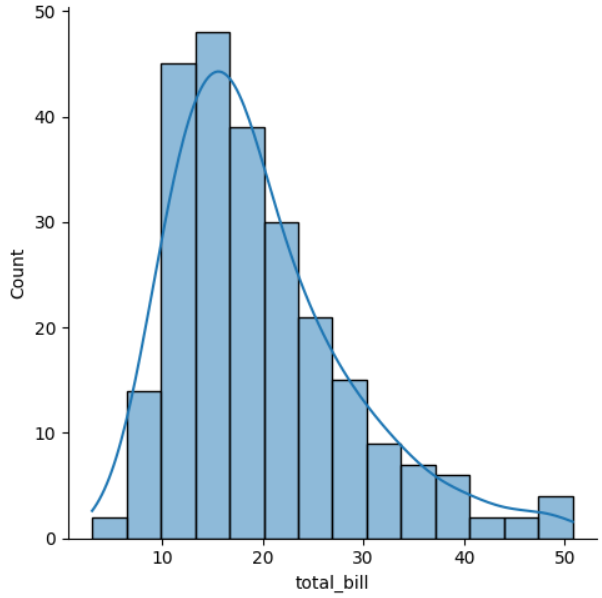
Description: Understand the importance of EDA-Quantitative and Qualitative analysis.

Code:

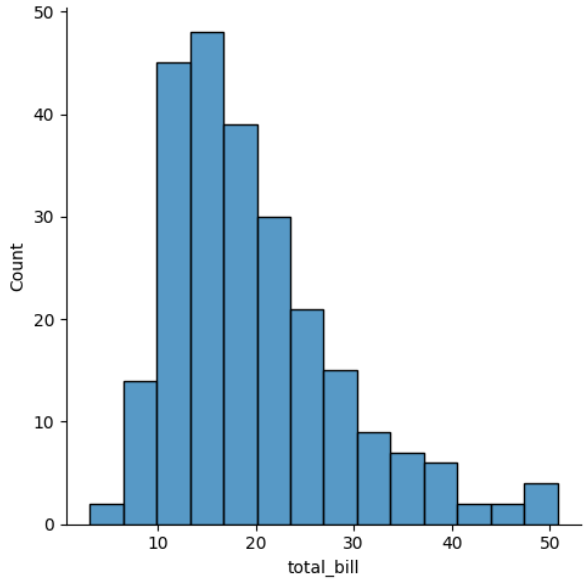
```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
tips.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

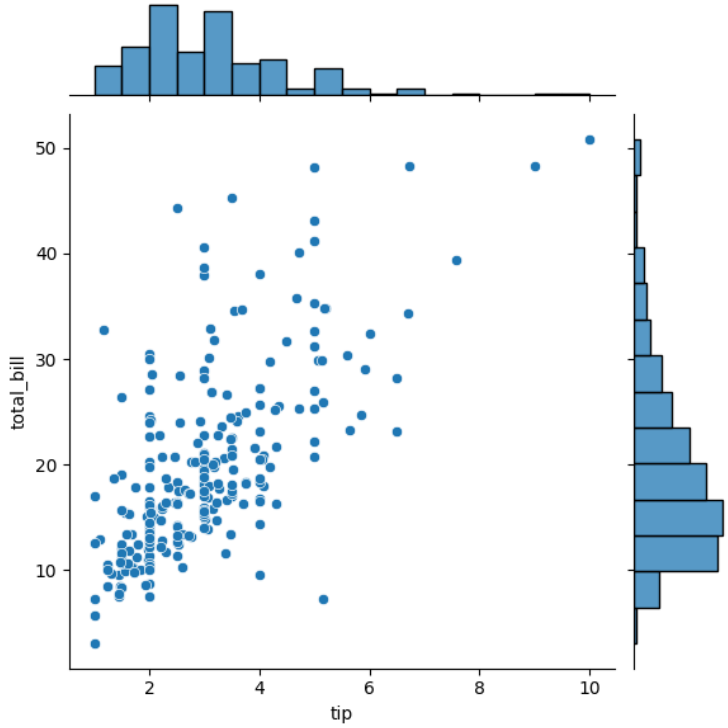
```
sns.displot(tips.total_bill,kde=True)
<seaborn.axisgrid.FacetGrid at 0x7ce42ed0dd30>
```



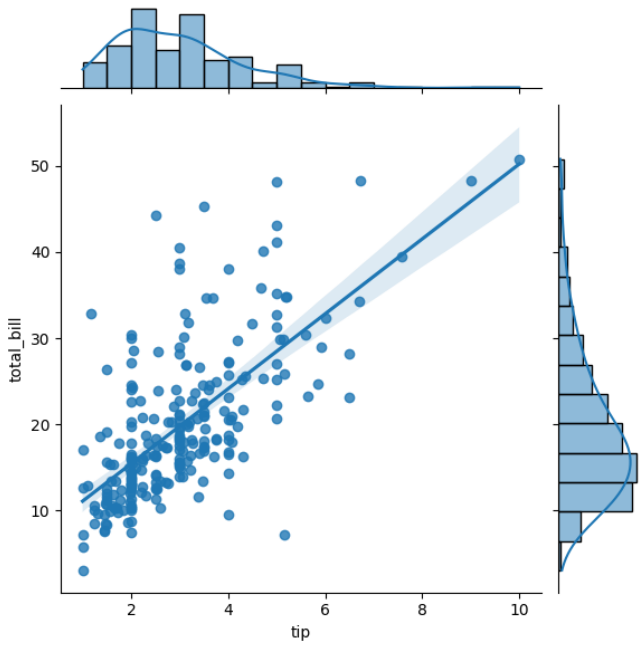

```
sns.displot(tips.total_bill,kde=False)
<seaborn.axisgrid.FacetGrid at 0x7ce42ead0910>
```



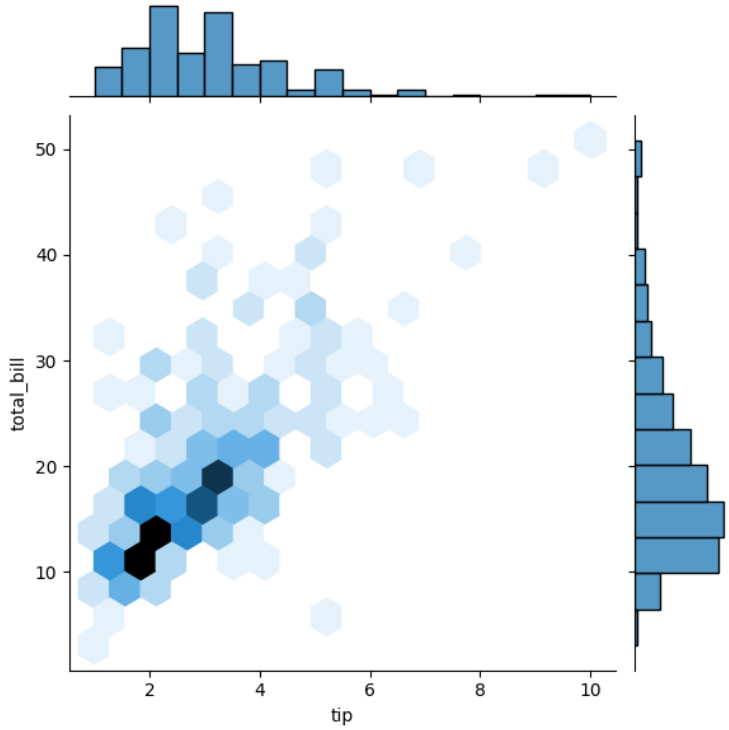
sns.jointplot(x=tips.tip,y=tips.total_bill)
<seaborn.axisgrid.JointGrid at 0x7ce42eb916a0>



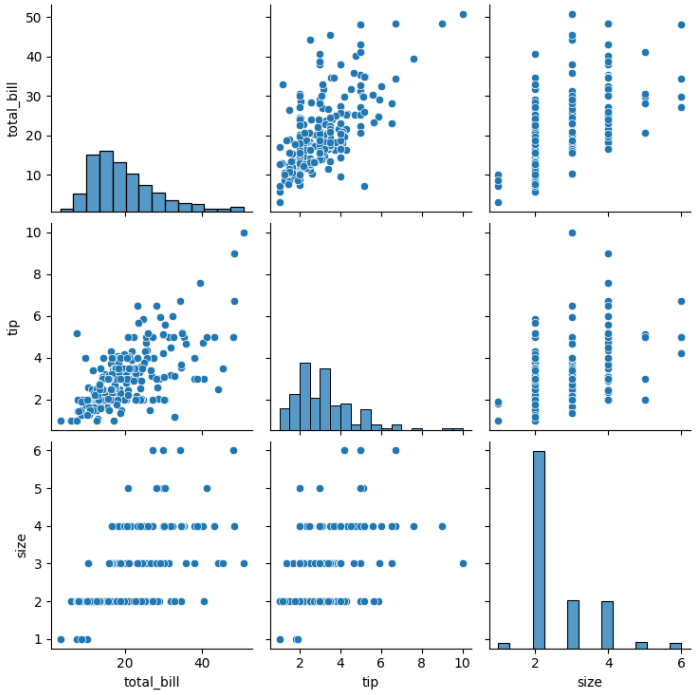
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
<seaborn.axisgrid.JointGrid at 0x7ce42a89d950>



sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
<seaborn.axisgrid.JointGrid at 0x7ce42a7c3390>



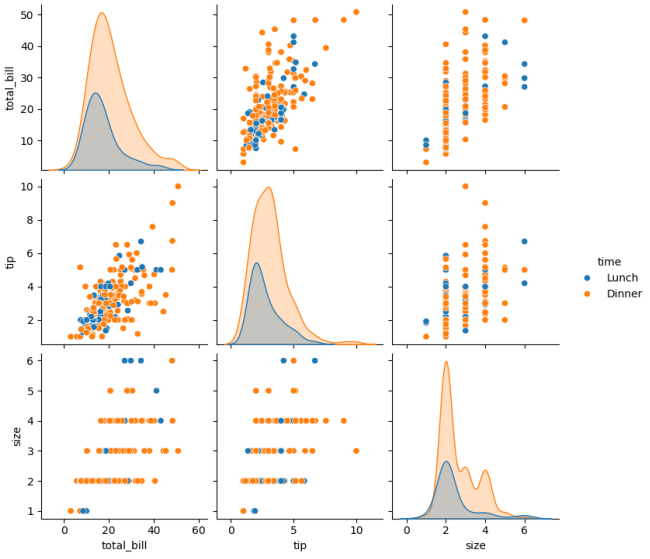
sns.pairplot(tips)
<seaborn.axisgrid.PairGrid at 0x7ce42eb92900>



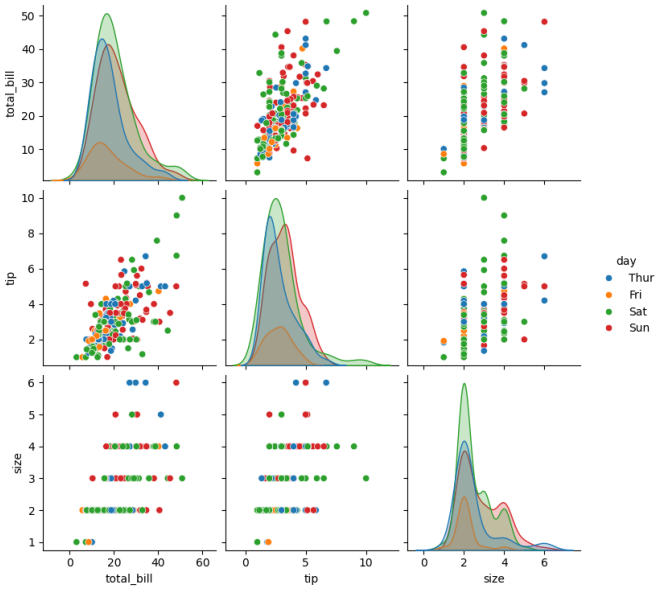
tips.time.value_counts()

	count
time	
Dinner	176
Lunch	68

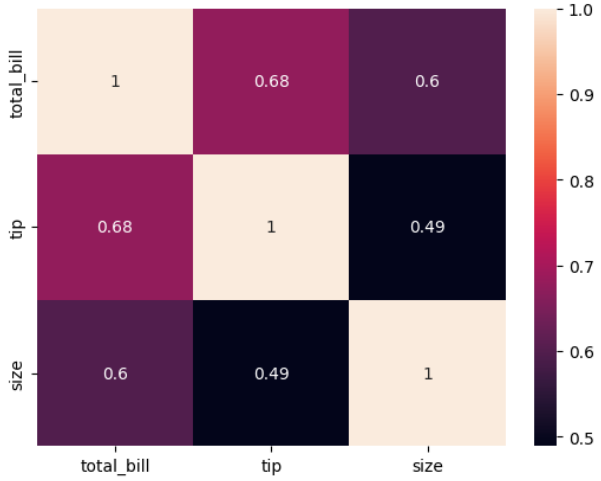
dtype: int64
sns.pairplot(tips,hue='time')
<seaborn.axisgrid.PairGrid at 0x7ce42859ead0>



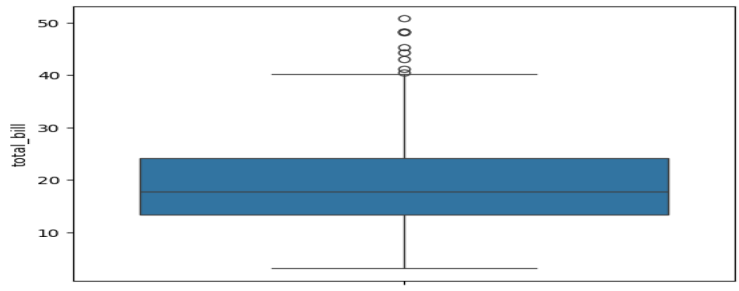
sns.pairplot(tips,hue='day')
<seaborn.axisgrid.PairGrid at 0x7ce423d83250>



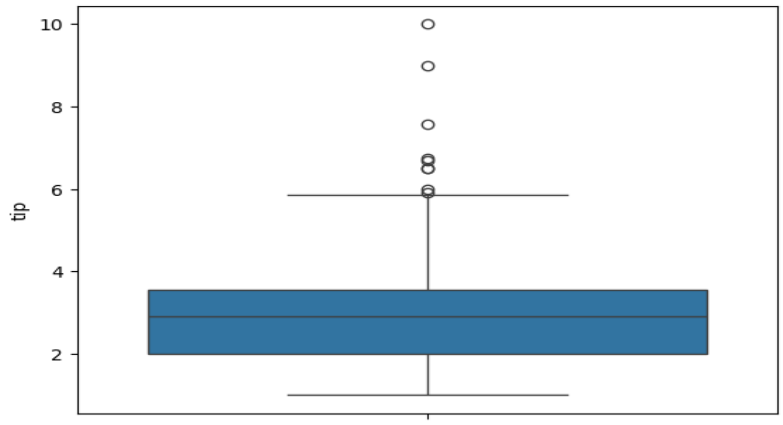
sns.heatmap(tips.corr(numeric_only=True),annot=True)
<Axes: >



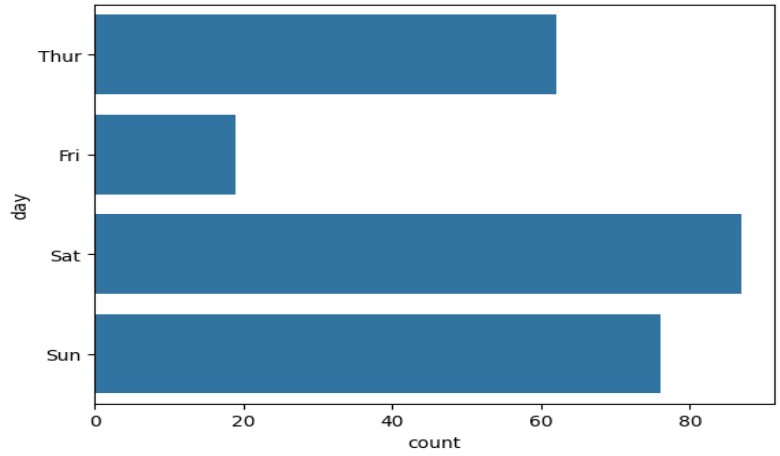
sns.boxplot(tips.total_bill)
<Axes: ylabel='total_bill'>



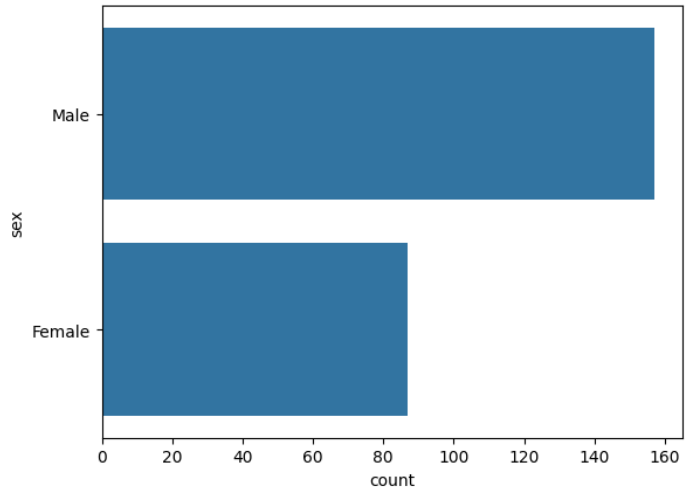
sns.boxplot(tips.tip)
<Axes: ylabel='tip'>



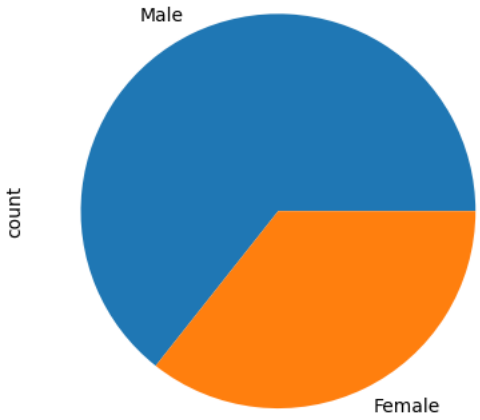
sns.countplot(tips.day)
<Axes: xlabel='count', ylabel='day'>



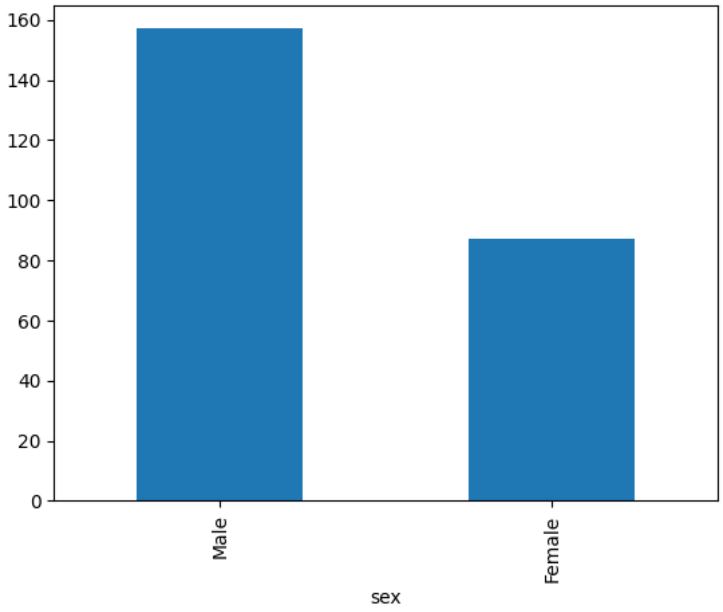
sns.countplot(tips.sex)
<Axes: xlabel='count', ylabel='sex'>



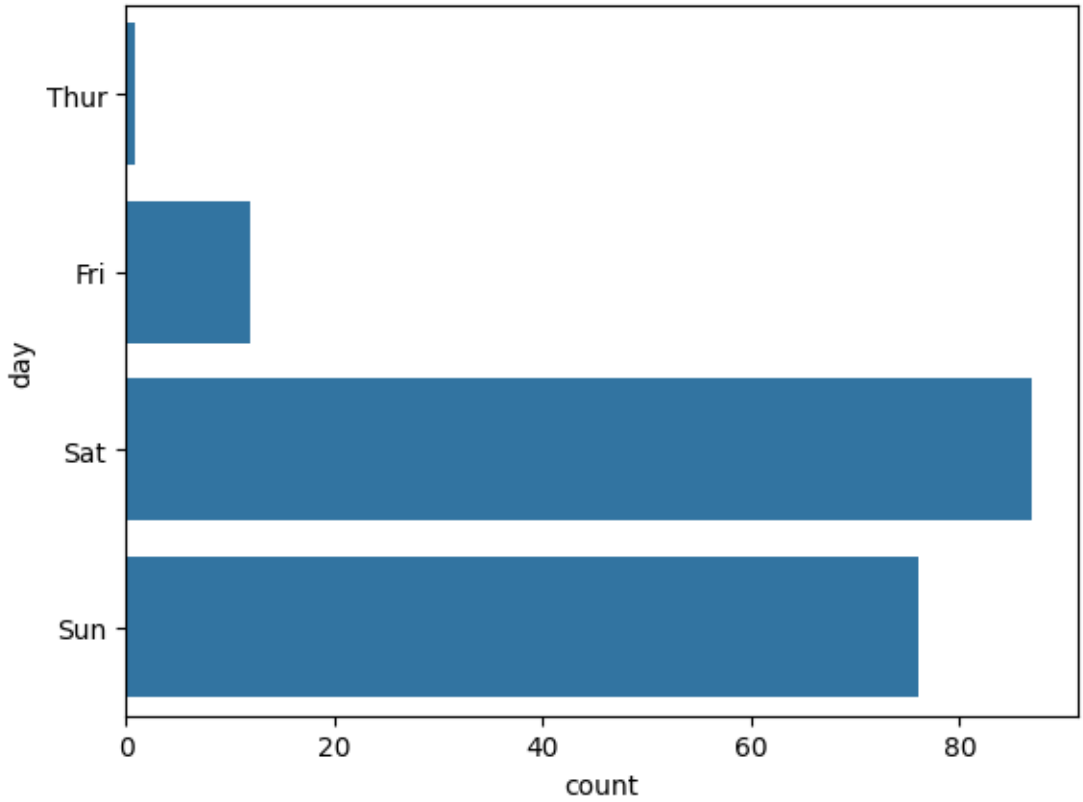
tips.sex.value_counts().plot(kind='pie')
<Axes: ylabel='count'>



tips.sex.value_counts().plot(kind='bar')
<Axes: xlabel='sex'>



sns.countplot(tips[tips.time=='Dinner']['day'])
<Axes: xlabel='count', ylabel='day'>



Result:
Understood the importance of EDA-Quantitative and Qualitative analysis.

Exp No:17 Experiment to understand Linear Regression for a given data set.

Description: Understand the Linear regression for the dataset given.

Code:

```
import numpy as np
import pandas as pd
df=pd.read_csv('/content/Salary_Data.csv')
df.head()
```

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary          30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
df.dropna(inplace=True)
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary          30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
df.describe()
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.556667	78345.133333
std	3.019669	28712.828489
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	5.000000	66983.500000
75%	8.125000	104512.000000
max	11.200000	127345.000000

```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
```

```
model.fit(x_train,y_train)
model.score(x_train,y_train)
0.9622248020359685
model.score(x_test,y_test)
0.958069463205837
model.coef_
array([[9343.7944329]])
model.intercept_
array([26528.39510872])
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}: ".format(yr_of_exp, Salary))
Enter Years of Experience: 44
Estimated Salary for 44.0 years of experience is [[437655.35015639]]:
```

Result:
Understood the Linear regression for the dataset given

Exp No:18 Experiment to understand KNN algorithm for a given data set

Description: Understand the KNN algorithm for the dataset given.

Code:

```
import numpy as np
import pandas as pd
df=pd.read_csv('/content/Iris.csv')
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
df.species.value_counts()
```

	count
species	
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

```
dtype: int64
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain, xtest, ytrain, ytest = train_test_split(features, label, test_size=0.2, random_state=42)

model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
KNeighborsClassifier()
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
0.9666666666666667
1.0
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])
```

Result:

Understood the KNN algorithm for the given dataset.

Exp No:19Experiment to understand Logistic Regression for a given data set.

Description: Understand the Logistic Regression algorithm for the dataset given.

Code:

```
import numpy as np
import pandas as pd
df=pd.read_csv('/content/Social_Network_Ads.csv')
df
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15600000	Male	41	125000	1
1	15600001	Female	28	140000	1
2	15600002	Male	25	75000	0
3	15600003	Male	53	35000	1
4	15600004	Male	55	35000	1
...	...	...	...	...	...
395	15600395	Female	22	85000	0
396	15600396	Female	50	85000	1
397	15600397	Female	18	50000	1
398	15600398	Female	35	100000	1
399	15600399	Male	49	115000	1

400 rows × 5 columns

```
df.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15600000	Male	41	125000	1
1	15600001	Female	28	140000	1
2	15600002	Male	25	75000	0
3	15600003	Male	53	35000	1
4	15600004	Male	55	35000	1

```
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
```

```
array([[    41, 125000],
       [    28, 140000],
       [    25,  75000],
       [    53,  35000],
       [    55,  35000],
       [    57,  20000],
       [    37,  85000],
       [    52,  60000],
       [    42,  30000],
       [    52,  90000],
       [    42, 120000],
       [    46,  20000],
       [    35, 145000],
       [    35,  15000],
       [    19,  25000],
       [    52, 110000],
       [    33,  95000],
       [    58,  65000],
       [    53,  35000],
       [    50,  35000],
       [    21,  25000],
       [    50,  15000],
       [    31, 135000],
       [    38,  50000],
       [    37,  60000],
       [    25,  65000],
       [    24,  70000],
       [    20,  35000],
       [    34,  50000],
       [    50, 120000],
```

```
[    29, 105000],
[    39,  50000],
[    39, 135000],
[    47,  25000],
[    55,  75000],
[    55,  60000],
[    25, 105000],
[    44,  50000],
[    44, 130000],
[    51,  85000],
[    38, 120000],
[    47, 140000],
[    50,  35000],
[    45,  50000],
[    50,  30000],
[    22,  30000],
[    36,  55000],
[    21,  15000],
[    52,  50000],
[    34,  90000],
[    45,  65000],
[    47,  70000],
[    46,  95000],
[    23,  80000],
[    52,  90000],
[    58,  20000],
[    54,  70000],
[    41,  40000],
[    46,  90000],
[    48, 105000],
[    52,  65000],
[    50, 110000],
[    38,  55000],
[    49,  85000],
[    40,  55000],
[    50, 115000],
[    20,  70000],
[    35,  30000],
[    42,  70000],
[    59, 140000],
[    48, 115000],
[    20,  85000],
[    57,  25000],
[    41,  20000],
[    49,  80000],
[    39, 115000],
[    40,  60000],
[    19,  35000],
[    44,  70000],
[    59, 140000],
[    19, 120000],
[    43, 115000],
[    34,  80000],
[    57, 130000],
[    50,  90000],
[    26,  40000],
[    56, 135000],
[    46,  70000],
[    59, 125000],
[    43, 115000],
[    52,  50000],
[    42,  35000],
[    41,  90000],
[    30,  50000],
[    24, 145000],
[    53, 115000],
[    37,  65000],
[    18, 120000],
[    25, 105000],
[    33, 125000],
[    31,  35000],
[    29,  35000],
[    40,  35000],
[    32, 130000],
[    45, 110000],
```

[51, 75000],
[19, 140000],
[49, 65000],
[40, 145000],
[39, 145000],
[42, 125000],
[39, 140000],
[39, 40000],
[59, 85000],
[23, 80000],
[32, 55000],
[54, 120000],
[50, 15000],
[25, 30000],
[22, 95000],
[56, 95000],
[21, 50000],
[23, 80000],
[49, 130000],
[47, 120000],
[52, 100000],
[57, 120000],
[33, 45000],
[30, 60000],
[59, 85000],
[47, 125000],
[36, 15000],
[34, 65000],
[36, 110000],
[45, 60000],
[43, 15000],
[54, 115000],
[43, 145000],
[40, 120000],
[26, 120000],
[29, 20000],
[18, 95000],
[18, 140000],
[51, 135000],
[49, 40000],
[42, 40000],
[57, 25000],
[18, 85000],
[33, 95000],
[56, 35000],
[22, 85000],
[39, 115000],
[46, 60000],
[20, 140000],
[29, 90000],
[43, 90000],
[33, 30000],
[54, 125000],
[39, 15000],
[46, 130000],
[31, 130000],
[45, 15000],
[22, 85000],
[47, 90000],
[22, 135000],
[29, 15000],
[33, 90000],
[43, 20000],
[43, 30000],
[38, 135000],
[56, 55000],
[53, 135000],
[50, 145000],
[47, 140000],
[54, 125000],
[40, 25000],
[27, 65000],
[22, 15000],
[53, 25000],
[51, 90000],

[48, 60000],
[27, 100000],
[36, 110000],
[49, 55000],
[18, 75000],
[22, 105000],
[21, 140000],
[33, 80000],
[41, 115000],
[33, 90000],
[19, 55000],
[45, 15000],
[49, 15000],
[44, 35000],
[37, 45000],
[41, 120000],
[29, 60000],
[52, 35000],
[50, 135000],
[50, 80000],
[54, 95000],
[29, 100000],
[20, 145000],
[18, 50000],
[50, 130000],
[57, 20000],
[27, 20000],
[46, 145000],
[30, 120000],
[29, 105000],
[48, 90000],
[19, 110000],
[52, 55000],
[40, 120000],
[34, 95000],
[43, 25000],
[25, 15000],
[46, 55000],
[43, 140000],
[27, 80000],
[43, 115000],
[51, 95000],
[58, 85000],
[24, 135000],
[21, 55000],
[28, 100000],
[46, 20000],
[53, 70000],
[42, 135000],
[38, 100000],
[53, 100000],
[27, 30000],
[54, 145000],
[26, 140000],
[41, 35000],
[52, 25000],
[52, 55000],
[53, 50000],
[35, 85000],
[56, 115000],
[49, 80000],
[41, 25000],
[40, 140000],
[49, 105000],
[54, 25000],
[29, 80000],
[30, 55000],
[40, 65000],
[42, 110000],
[52, 95000],
[58, 35000],
[47, 95000],
[34, 135000],
[37, 85000],
[42, 20000],

[39, 70000],
[30, 40000],
[36, 145000],
[53, 110000],
[29, 65000],
[58, 25000],
[36, 110000],
[29, 80000],
[26, 55000],
[24, 75000],
[45, 45000],
[31, 15000],
[48, 115000],
[36, 80000],
[33, 30000],
[22, 70000],
[52, 45000],
[29, 115000],
[42, 60000],
[38, 140000],
[53, 85000],
[40, 140000],
[33, 75000],
[56, 55000],
[59, 140000],
[56, 35000],
[31, 75000],
[48, 140000],
[22, 25000],
[52, 95000],
[40, 115000],
[46, 145000],
[28, 35000],
[35, 90000],
[29, 135000],
[26, 145000],
[27, 85000],
[34, 90000],
[55, 140000],
[24, 25000],
[30, 90000],
[57, 70000],
[59, 145000],
[26, 130000],
[44, 90000],
[19, 105000],
[22, 145000],
[46, 85000],
[54, 140000],
[55, 120000],
[36, 30000],
[25, 105000],
[18, 40000],
[39, 50000],
[34, 75000],
[24, 65000],
[42, 115000],
[21, 15000],
[53, 65000],
[23, 40000],
[48, 120000],
[36, 70000],
[56, 145000],
[44, 45000],
[27, 90000],
[43, 35000],
[36, 145000],
[56, 135000],
[20, 130000],
[30, 30000],
[45, 115000],
[37, 130000],
[45, 80000],
[25, 70000],
[58, 70000],

```
       [   56,  50000],
       [   18, 110000],
       [   20,  25000],
       [   30,  55000],
       [   45,  75000],
       [   42,  20000],
       [   50, 140000],
       [   55,  20000],
       [   23,  70000],
       [   49,  50000],
       [   38,  55000],
       [   33, 140000],
       [   38,  80000],
       [   28,  30000],
       [   54,  60000],
       [   53, 135000],
       [   52, 105000],
       [   36,  15000],
       [   37,  30000],
       [   35, 115000],
       [   58,  85000],
       [   31,  60000],
       [   32,  30000],
       [   48,  65000],
       [   18, 110000],
       [   20,  75000],
       [   33,  15000],
       [   40, 105000],
       [   28, 145000],
       [   29,  35000],
       [   27, 105000],
       [   49,  15000],
       [   33, 145000],
       [   25,  70000],
       [   55, 105000],
       [   29,  85000],
       [   41, 105000],
       [   45,  75000],
       [   25,  50000],
       [   45, 130000],
       [   53,  50000],
       [   43,  40000],
       [   25,  25000],
       [   45,  50000],
       [   45, 130000],
       [   54,  80000],
       [   58,  65000],
       [   53, 140000],
       [   44,  25000],
       [   34,  55000],
       [   26,  60000],
       [   50,  90000],
       [   37, 115000],
       [   30, 145000],
       [   45,  90000],
       [   46, 135000],
       [   30, 140000],
       [   52, 120000],
       [   23,  40000],
       [   35, 140000],
       [   22, 120000],
       [   42, 125000],
       [   19,  45000],
       [   27,  90000],
       [   47,  75000],
       [   22,  85000],
       [   50,  85000],
       [   18,  50000],
       [   35, 100000],
       [   49, 115000]])
label
array([1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1,
```

```
    1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1,
    1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1,
    1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1,
    1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1,
    1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1,
    1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0,
    0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
    1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1,
    1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
    1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0,
    1, 1, 1, 1])
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load dataset and define features and label
df = pd.read_csv('Social_Network_Ads.csv')
features = df[['Age', 'EstimatedSalary']]
label = df['Purchased']

for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_spec = train_test_split(
        features, label, test_size=0.25, random_state=i
    )
    model = LogisticRegression()
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(test_score, train_score, i))
```

```
Test 0.81 Train 0.7933333333333333 Random State 2
Test 0.81 Train 0.8033333333333333 Random State 4
Test 0.79 Train 0.7866666666666666 Random State 6
Test 0.81 Train 0.8066666666666666 Random State 9
Test 0.83 Train 0.8033333333333333 Random State 10
Test 0.82 Train 0.81 Random State 16
Test 0.82 Train 0.8033333333333333 Random State 19
Test 0.82 Train 0.7866666666666666 Random State 22
Test 0.85 Train 0.7866666666666666 Random State 29
Test 0.82 Train 0.8 Random State 30
Test 0.82 Train 0.8066666666666666 Random State 31
Test 0.82 Train 0.8 Random State 36
Test 0.86 Train 0.7933333333333333 Random State 38
Test 0.88 Train 0.7733333333333333 Random State 40
Test 0.85 Train 0.7933333333333333 Random State 41
Test 0.82 Train 0.8033333333333333 Random State 46
Test 0.84 Train 0.79 Random State 51
Test 0.87 Train 0.7866666666666666 Random State 53
Test 0.81 Train 0.8 Random State 54
Test 0.82 Train 0.7966666666666666 Random State 55
Test 0.82 Train 0.7966666666666666 Random State 56
Test 0.81 Train 0.8066666666666666 Random State 57
Test 0.82 Train 0.7933333333333333 Random State 58
Test 0.86 Train 0.7833333333333333 Random State 62
Test 0.81 Train 0.8 Random State 63
Test 0.83 Train 0.8066666666666666 Random State 65
Test 0.81 Train 0.7966666666666666 Random State 68
Test 0.83 Train 0.8 Random State 69
Test 0.84 Train 0.7966666666666666 Random State 70
Test 0.82 Train 0.7966666666666666 Random State 71
Test 0.84 Train 0.79 Random State 73
Test 0.85 Train 0.79 Random State 75
Test 0.85 Train 0.7833333333333333 Random State 76
```

Test 0.84 Train 0.7966666666666666 Random State 77
Test 0.81 Train 0.8033333333333333 Random State 80
Test 0.81 Train 0.78 Random State 81
Test 0.8 Train 0.7866666666666666 Random State 84
Test 0.85 Train 0.79 Random State 85
Test 0.84 Train 0.7966666666666666 Random State 86
Test 0.82 Train 0.8033333333333333 Random State 88
Test 0.83 Train 0.8 Random State 90
Test 0.84 Train 0.79 Random State 92
Test 0.85 Train 0.7833333333333333 Random State 98
Test 0.83 Train 0.7966666666666666 Random State 99
Test 0.81 Train 0.8066666666666666 Random State 101
Test 0.84 Train 0.7833333333333333 Random State 102
Test 0.84 Train 0.79 Random State 107
Test 0.82 Train 0.7766666666666666 Random State 108
Test 0.81 Train 0.7933333333333333 Random State 110
Test 0.81 Train 0.8066666666666666 Random State 115
Test 0.86 Train 0.7933333333333333 Random State 116
Test 0.82 Train 0.8033333333333333 Random State 117
Test 0.85 Train 0.79 Random State 118
Test 0.84 Train 0.7966666666666666 Random State 119
Test 0.82 Train 0.8033333333333333 Random State 124
Test 0.81 Train 0.8033333333333333 Random State 125
Test 0.82 Train 0.8 Random State 126
Test 0.87 Train 0.79 Random State 127
Test 0.82 Train 0.8 Random State 128
Test 0.83 Train 0.8 Random State 129
Test 0.83 Train 0.8 Random State 131
Test 0.81 Train 0.8033333333333333 Random State 134
Test 0.84 Train 0.8 Random State 136
Test 0.83 Train 0.8033333333333333 Random State 137
Test 0.84 Train 0.8 Random State 142
Test 0.84 Train 0.7866666666666666 Random State 145
Test 0.81 Train 0.8033333333333333 Random State 146
Test 0.82 Train 0.79 Random State 149
Test 0.81 Train 0.8066666666666666 Random State 154
Test 0.84 Train 0.8 Random State 155
Test 0.84 Train 0.7866666666666666 Random State 156
Test 0.83 Train 0.8 Random State 157
Test 0.83 Train 0.7866666666666666 Random State 158
Test 0.87 Train 0.7833333333333333 Random State 160
Test 0.87 Train 0.78 Random State 162
Test 0.81 Train 0.8066666666666666 Random State 164
Test 0.81 Train 0.7866666666666666 Random State 165
Test 0.82 Train 0.8 Random State 166
Test 0.81 Train 0.8 Random State 169
Test 0.81 Train 0.8033333333333333 Random State 172
Test 0.83 Train 0.8066666666666666 Random State 173
Test 0.81 Train 0.8 Random State 176
Test 0.86 Train 0.79 Random State 177
Test 0.84 Train 0.7966666666666666 Random State 178
Test 0.85 Train 0.7833333333333333 Random State 180
Test 0.83 Train 0.8033333333333333 Random State 181
Test 0.84 Train 0.79 Random State 183
Test 0.82 Train 0.8 Random State 186
Test 0.83 Train 0.7966666666666666 Random State 187
Test 0.83 Train 0.7933333333333333 Random State 188
Test 0.86 Train 0.7833333333333333 Random State 192
Test 0.81 Train 0.8066666666666666 Random State 193
Test 0.84 Train 0.8 Random State 194
Test 0.82 Train 0.79 Random State 195
Test 0.82 Train 0.7966666666666666 Random State 198
Test 0.83 Train 0.8 Random State 200
Test 0.84 Train 0.7966666666666666 Random State 203
Test 0.81 Train 0.8066666666666666 Random State 211
Test 0.83 Train 0.8 Random State 212
Test 0.84 Train 0.7866666666666666 Random State 213
Test 0.83 Train 0.7933333333333333 Random State 216
Test 0.88 Train 0.7866666666666666 Random State 217
Test 0.81 Train 0.8033333333333333 Random State 218
Test 0.82 Train 0.8033333333333333 Random State 221
Test 0.81 Train 0.8066666666666666 Random State 224
Test 0.82 Train 0.8 Random State 232
Test 0.84 Train 0.7966666666666666 Random State 233
Test 0.82 Train 0.7933333333333333 Random State 238

Test 0.83 Train 0.7933333333333333 Random State 242
Test 0.82 Train 0.8 Random State 244
Test 0.83 Train 0.79 Random State 247
Test 0.83 Train 0.7833333333333333 Random State 250
Test 0.82 Train 0.8033333333333333 Random State 253
Test 0.82 Train 0.8033333333333333 Random State 254
Test 0.83 Train 0.8 Random State 255
Test 0.8 Train 0.78 Random State 257
Test 0.84 Train 0.7833333333333333 Random State 258
Test 0.84 Train 0.8033333333333333 Random State 263
Test 0.82 Train 0.8 Random State 264
Test 0.81 Train 0.8 Random State 267
Test 0.85 Train 0.8 Random State 268
Test 0.82 Train 0.8033333333333333 Random State 269
Test 0.81 Train 0.8 Random State 271
Test 0.88 Train 0.79 Random State 275
Test 0.81 Train 0.7966666666666666 Random State 276
Test 0.82 Train 0.79 Random State 278
Test 0.83 Train 0.7933333333333333 Random State 280
Test 0.84 Train 0.7966666666666666 Random State 284
Test 0.81 Train 0.7966666666666666 Random State 285
Test 0.81 Train 0.8066666666666666 Random State 286
Test 0.84 Train 0.8 Random State 289
Test 0.84 Train 0.7933333333333333 Random State 294
Test 0.85 Train 0.7866666666666666 Random State 295
Test 0.83 Train 0.8066666666666666 Random State 297
Test 0.85 Train 0.7933333333333333 Random State 301
Test 0.82 Train 0.7966666666666666 Random State 306
Test 0.85 Train 0.7933333333333333 Random State 308
Test 0.82 Train 0.7966666666666666 Random State 309
Test 0.87 Train 0.7833333333333333 Random State 310
Test 0.84 Train 0.7966666666666666 Random State 311
Test 0.82 Train 0.8 Random State 312
Test 0.82 Train 0.8066666666666666 Random State 316
Test 0.81 Train 0.8066666666666666 Random State 317
Test 0.81 Train 0.8066666666666666 Random State 318
Test 0.81 Train 0.8066666666666666 Random State 319
Test 0.84 Train 0.79 Random State 320
Test 0.85 Train 0.7866666666666666 Random State 324
Test 0.87 Train 0.79 Random State 329
Test 0.84 Train 0.8 Random State 330
Test 0.83 Train 0.8033333333333333 Random State 333
Test 0.84 Train 0.7933333333333333 Random State 334
Test 0.83 Train 0.79 Random State 337
Test 0.82 Train 0.78 Random State 339
Test 0.82 Train 0.8 Random State 340
Test 0.82 Train 0.81 Random State 341
Test 0.87 Train 0.7833333333333333 Random State 344
Test 0.82 Train 0.8066666666666666 Random State 346
Test 0.84 Train 0.8033333333333333 Random State 347
Test 0.84 Train 0.7866666666666666 Random State 348
Test 0.84 Train 0.7833333333333333 Random State 350
Test 0.83 Train 0.8033333333333333 Random State 352
Test 0.84 Train 0.7933333333333333 Random State 353
Test 0.82 Train 0.8033333333333333 Random State 354
Test 0.83 Train 0.7866666666666666 Random State 355
Test 0.89 Train 0.77 Random State 356
Test 0.82 Train 0.7966666666666666 Random State 359
Test 0.83 Train 0.7966666666666666 Random State 360
Test 0.85 Train 0.7933333333333333 Random State 368
Test 0.83 Train 0.7966666666666666 Random State 369
Test 0.86 Train 0.79 Random State 370
Test 0.81 Train 0.8066666666666666 Random State 372
Test 0.82 Train 0.8 Random State 374
Test 0.84 Train 0.8 Random State 378
Test 0.87 Train 0.7866666666666666 Random State 379
Test 0.84 Train 0.7933333333333333 Random State 381
Test 0.81 Train 0.8 Random State 383
Test 0.81 Train 0.7966666666666666 Random State 384
Test 0.82 Train 0.8066666666666666 Random State 385
Test 0.85 Train 0.7933333333333333 Random State 389
Test 0.82 Train 0.8066666666666666 Random State 394
Test 0.81 Train 0.8066666666666666 Random State 397

```
x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=42)
```

```
# Train the model
finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
0.83125
0.7125
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
              precision    recall  f1-score   support

           0       0.67      0.48      0.55       101
           1       0.84      0.92      0.88       299

    accuracy                           0.81       400
   macro avg       0.75      0.70      0.72       400
weighted avg       0.80      0.81      0.80       400
```

Result:

Understood the Logistic Regression algorithm for the dataset given.

Exp No:20 Experiment to understand K-means clustering algorithm for a given data set.

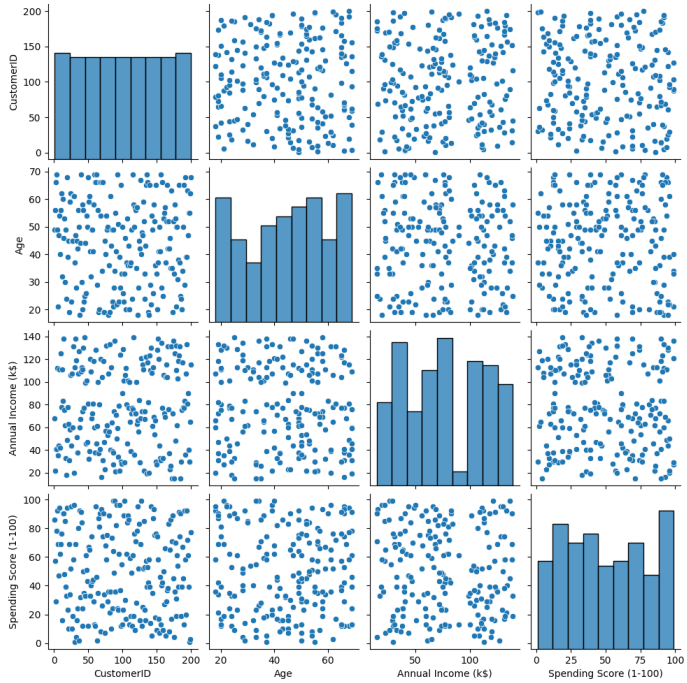
Description: Understand the K-means clustering algorithm for the dataset given.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv('/content/Mall_Customers.csv')
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	49	68	86
1	2	Female	56	22	57
2	3	Male	66	41	29
3	4	Male	69	41	78
4	5	Male	49	112	92

```
sns.pairplot(df)
<seaborn.axisgrid.PairGrid at 0x7841ea0a0ad0>
```

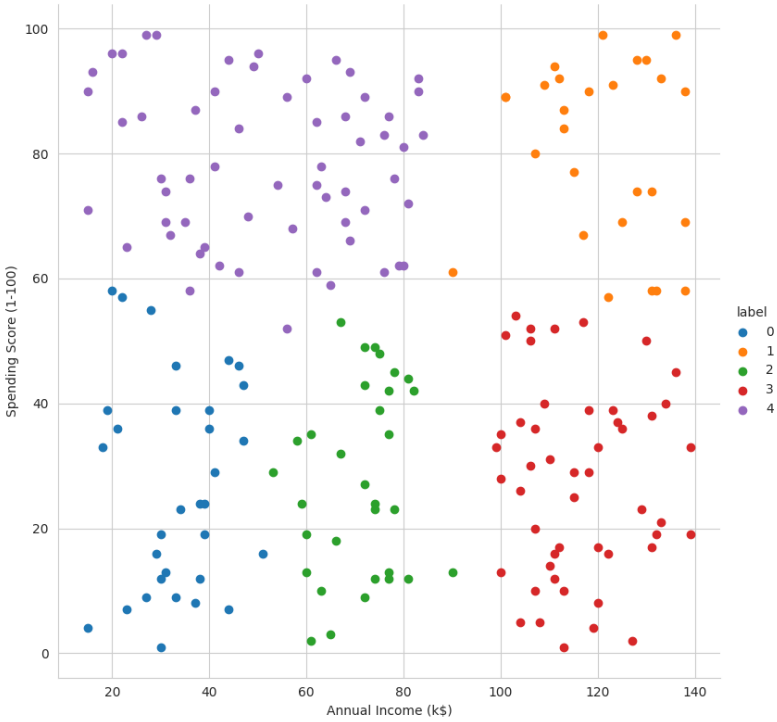



```
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
/tmp/ipykernel_2901/470183701.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

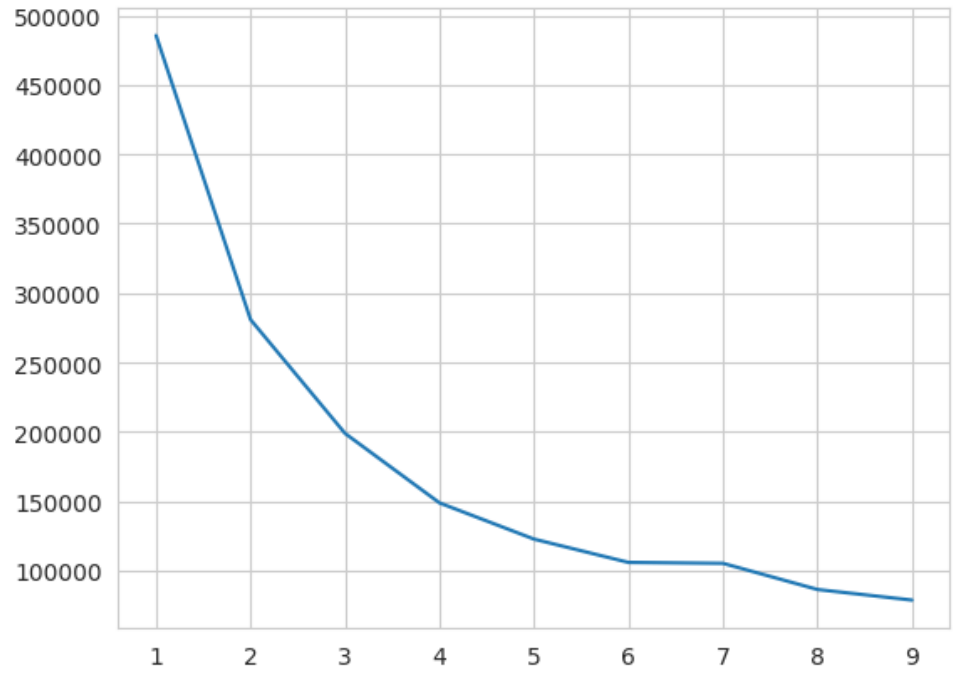
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  Final['label']=model.predict(features)
```

	Annual Income (k$)	Spending Score (1-100)	label
0	68	86	4
1	22	57	0
2	41	29	0
3	41	78	4
4	112	92	1

```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```




```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
 model=KMeans(n_clusters=i)
 model.fit(features_el)
 wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
[<matplotlib.lines.Line2D at 0x7841dd2a0e10>]
```



Result:
Understood the K-means clustering algorithm for the given dataset.